KShieldVPN

Submitted In Partial Fulfillment of Requirements
For the Degree Of

Bachelor of Science
Computer Science (Hons.)

By

Kunal Ravindra Patil

Roll No: 31011122061

Guide

Mr. Shrinivas Acharya

S K Somaiya College

Somaiya Vidyavihar University

Vidyavihar, Mumbai - 400 077

2022-25

Somaiya Vidyavihar University

S K Somaiya College

Certificate

This is to certify that the project report entitled KShieldVPN is bonafide record of the project work done by **Kunal Ravindra Patil** in the year 2024-25 under the guidance of **Mr. Shrinivas Acharya** Department of Information Technology and Computer Science in partial fulfillment of requirement for the Bachelor of Science in Computer Science (Hons) degree in S K Somaiya College of Somaiya Vidyavihar University.

Project Guide	Programme Coordinator
Head of Department	Dr Neetin Desai Dean and Professor

Date:

Place: Mumbai-77

Somaiya Vidyavihar University

S K Somaiya College

Certificate of Approval of Examiners

This is to certify that the project report entitled **KShieldVPN** is bonafide record of the project work done by **Kunal Ravindra Patil** in partial fulfillment of requirement for the Bachelor of Science in Computer Science (Hons) degree in S K Somaiya College of Somaiya Vidyavihar University.

_____ _____

External Examiner /Expert Internal Examiner/ Guide

Date:

Place: Mumbai-77

Somaiya Vidyavihar University

S K Somaiya College

DECLARATION

I declare that this written report submission represents the work done based on my and / or others' ideas with adequately cited and referenced the original source. I also declare that I have adhered to all principles of academic honesty and integrity as I have not misinterpreted or fabricated or falsified any idea/data/fact/source/original work/ matter in my submission.

I understand that any violation of the above will be cause for disciplinary action by the college and may evoke the penal action from the sources which have not been properly cited or from whom proper permission is not sought.

Signature of the Student

Kunal Ravindra Patil
Name of the Student

31011122061
Roll No.

Date:

Place: Mumbai-77

Abstract

With the increasing concerns over online privacy and data security, the need for secure communication channels has become essential in today's world which relies heavily on internet . The project, KShieldVPN, aims to develop a reliable and user-friendly Virtual Private Network (VPN) solution to enhance data security, privacy, and anonymity for users. It primarily focuses on providing a secure browsing experience.

KShieldVPN is designed with a simple and intuitive user interface ensuring ease of use even for non-technical users. The project integrates various security features, such as AES encryption, secure authentication, and IP masking, to mitigate potential cyber threats and ensure a seamless connection experience.

The development process involved designing an efficient UI/UX, implementing robust encryption techniques, and conducting extensive testing. Testing results indicate improvements in network security and user experience, demonstrating the application's effectiveness in securing online communications.

By providing a secure, efficient, and easy-to-use VPN service, KShieldVPN serves as a valuable tool for individuals and organizations seeking reliable protection against cyber threats, surveillance, and data breaches. This solution contributes to a safer digital environment, ensuring users can browse the internet with confidence and privacy.

Keywords: VPN(Virtual private network), encryption, Ip masking

Acknowledgement

I would like to take this opportunity to express my heartfelt gratitude to all those who have contributed to the successful completion of this project.

First and foremost, I am deeply thankful to my college for providing me with the platform and resources to pursue my studies and undertake this final project. The supportive academic environment has been instrumental in shaping my understanding and skills.

I extend my sincere appreciation to our esteemed Director, Dr Niteen Desai, for his constant encouragement and leadership, which have inspired me throughout my journey. My heartfelt thanks also go to Dr. Swati Maurya, Head of the Information Technology and Computer Science Department, and Ms. Minal Dive Department Coordinator, for their invaluable guidance and support. Their insights and advice have been pivotal in navigating the challenges of this project.

I am especially grateful to my project guide, Mr. Shrinivas Acharya, for his exceptional mentorship. His expertise, patience, and constructive feedback have been invaluable in steering this project toward success. His dedication to providing the necessary resources and fostering a learning-oriented environment has greatly enhanced my experience.

I would also like to thank all the faculty members of the Information Technology and Computer Science Department for their encouragement and assistance, both directly and indirectly. Their collective knowledge and unwavering support have been a constant source of motivation.

Lastly, I owe immense gratitude to my parents and friends for their unconditional love, encouragement, and moral support. Their belief in my abilities and their steadfast presence during challenging times have been instrumental in helping me complete this endeavour.

This project would not have been possible without the contributions of each and every one mentioned above. I am truly grateful for their support and guidance throughout this journey.

Contents

Ch. No	Name of Chapter			Pg No
	List of figures			8
	List of Tables			9
1	**Introduction**			10
	1.1	Introduction		10
	1.2	Background		11
	1.3	Objectives		12
	1.4	Purpose, Scope and Applicability		13
2	**Survey of Technologies**			14
	2.1	Technologies implemented for the project		14
3	**Requirements and Analysis**			16
	3.1	Problem definition		16
	3.2	Requirements Specification		17
	3.3	SDLC Model		21
	3.4	Planning and Scheduling		23
	3.5	Software and Hardware Requirements		25
	3.6	Preliminary Product Description		27
	3.7	Conceptual Models		29
		3.7.1	Use Case Diagram	29
		3.7.2	DFD Level Diagram	30
		3.7.3	ER Diagram	31
		3.7.4	Class Diagram	32

		3.7.5	Activity Diagram	33
		3.7.6	Sequence Diagram	34
4	**System Design**			35
	4.1	Basic Modules		35
	4.2	Data Design		39
	4.3	Procedural Design		41
	4.4	User Interface Design		45
	4.5	Security Issues		52
	4.6	Test Case Design		54
5	**Implementation and testing**			56
	5.1	Implementation Approach		56
	5.2	Coding Details and Coding Efficiency		60
	5.3	Testing Approach		63
	5.4	Modifications and Improvements		69
	5.5	Test Cases		71
6	**Results and Discussion**			82
	6.1	Test Reports		82
	6.2	User Documentation		91
7	**Conclusions**			94
	7.1	Conclusion		94
	7.2	Limitations of the System		97
	7.3	Future Scope of the Project		98
References				99

List of Figures

Sr No	Name of figure	Pg no.
1	Figure 3.3 Agile Model	22
2	Figure 3.4 Gantt Chart	23
3	Figure 3.7.1 Use case Diagram	29
4	Figure 3.7.2 DFD level Diagram	30
5	Figure 3.7.3 ER Diagram	31
6	Figure 3.7.4 Class Diagram	32
7	Figure 3.7.5 Activity Diagram	33
9	Figure 3.7.6 Sequence Diagram	34
8	Figure 4.4.1 User Signup Design	47
9	Figure 4.4.2 User Login Design	47
10	Figure 4.4.3 Forgot password Design	48
11	Figure 4.4.4 Reset Password Design	48
12	Figure 4.4.5 Home Page Design	49
13	Figure 4.4.6 Speed Test Page Design	49
14	Figure 4.4.7 DNS Leak Test Design	49
15	Figure 4.4.8 Payment Gateway Design	50
16	Figure 4.4.9 Payment Successful Design	51
17	Figure 4.4.10 Payment Cancelled Design	51

List of Tables

Sr No	Name of Table	Pg no.
1	Client-side software requirements	25
2	Client-side hardware requirements	25
3	Developer side software requirements	26
4	Developer side hardware requirements	26
5	General test cases	71
6	Unit testing test cases	73
7	Integration testing test cases	77
8	System testing test cases	80

Chapter 1

Introduction

This chapter presents the idea about project, its introduction and background research, what problem the user faced and what are the objectives, scope, advantages and applicability of the project.

1.1 Introduction

In the current digital era, issues related to online privacy and security have risen to prominence due to a surge in cyber threats, data leaks, and governmental monitoring.

A Virtual Private Network (VPN) is a solution that bolsters security by encrypting internet traffic and concealing the user's IP address. By establishing a secure connection between the user's device and a distant server, a VPN obstructs unauthorized access, ensuring that sensitive data stays protected. This makes VPNs crucial for both individuals and organizations aiming to secure their online activities.

VPNs serve multiple purposes, including protecting data on public Wi-Fi networks, bypassing geo-restrictions, and enabling anonymous browsing. They are widely used by organizations to allow employees to access corporate resources securely from remote locations. VPNs operate using encryption protocols ensuring that transmitted data remains unreadable to hackers or third parties. With the rise of cybercrime and mass surveillance, the demand for reliable and user-friendly VPN solutions has grown significantly.

To address these concerns, KShieldVPN has been developed as a secure, efficient, and easy-to-use VPN solution. KShieldVPN not only helps in protecting user data but also enables secure access to restricted content, corporate networks, and region-locked services.

It is particularly useful for individuals, remote workers, and businesses that require confidential communication and unrestricted internet access. By combining strong encryption, ease of use, and high-speed performance, KShieldVPN provides a practical solution for online privacy and security, ensuring that users can browse the internet with confidence.

1.2 Background

The evolution of Virtual Private Networks (VPNs) is closely linked to the development of internet technologies aimed at enhancing security and privacy. Early internet protocols, such as TCP/IP, facilitated global communication but lacked robust mechanisms for protecting sensitive data.

As the internet grew in the 1980s and 1990s, vulnerabilities emerged, prompting researchers to develop encryption and tunnelling technologies. This led to the creation of early secure communication methods, laying the groundwork for modern VPN solutions.

The first significant advancement in VPN technology came with the introduction of SwIPe (Software IP Encryption Protocol) in 1993. Although not widely adopted due to its complexity, SwIPe established the foundational concept of creating secure tunnels for data transmission. In the mid-1990s, IPsec (Internet Protocol Security) emerged as a standardized solution that encrypted data at the network layer, allowing businesses to establish private communication channels over public networks. T

his marked a turning point in secure internet communication, enabling organizations to protect sensitive information.

As VPN technology evolved, Microsoft developed PPTP (Point-to-Point Tunneling Protocol), which allowed users to create secure connections over the internet. Cisco introduced L2F (Layer 2 Forwarding Protocol), facilitating secure data transmission but lacking encryption capabilities. The merging of these two protocols into L2TP (Layer 2 Tunneling Protocol) in 1999 combined connection stability with encryption features. These advancements set the stage for modern VPNs that prioritize security and performance.

The introduction of OpenVPN in 2001 revolutionized VPN technology by offering an open-source solution with strong encryption based on SSL/TLS protocols. OpenVPN's flexibility and ability to bypass firewalls made it superior to earlier protocols like PPTP and L2TP. Today’s VPNs also utilize advanced protocols such as WireGuard, which provides faster speeds and stronger encryption compared to traditional options. These innovations reflect ongoing efforts to address evolving cybersecurity threats while maintaining user-friendliness.

KShieldVPN represents a modern solution built on this rich history of VPN technology. Designed with privacy, security, and ease of use in mind, KShieldVPN leverages OpenVPN’s robust encryption capabilities while offering an intuitive interface for seamless connectivity. It ensures safe browsing by encrypting user data and masking IP addresses, preventing tracking by ISPs and cybercriminals. Tailored for both individuals and businesses, KShieldVPN empowers users to navigate the internet securely while enjoying unrestricted access to global content.

1.3 Objectives

1. Ensure Secure Data Transmission – Implement end-to-end encryption using industry-standard cryptographic protocols to protect user data from interception, cyber threats, and unauthorized access. By securing all transmitted data, KShieldVPN aims to provide users with a safe and private browsing experience, even on unsecured networks.

2. Enhance Online Privacy and Anonymity – Conceal users' IP addresses and prevent tracking by ISPs, hackers, and third parties. By rerouting traffic through secure VPN servers, KShieldVPN ensures that users can browse anonymously and protect their digital footprint from surveillance or data collection.

3. Develop a User-Friendly Interface – Design and implement an intuitive, easy-to-navigate UI using Guna2 UI, ensuring accessibility for both technical and non-technical users. The application will provide simple controls, seamless configuration, and a clean design, making it convenient for users to connect to the VPN without extensive technical knowledge.

4. Enable Secure Remote Access – Allow users to securely connect to the internet and private networks from any location, especially when using public Wi-Fi or working remotely. This feature is essential for businesses, remote workers, and individuals who require confidential and protected communications over untrusted networks.

5. Support Reliable and High-Speed Connectivity – Optimize VPN server connections to ensure minimal speed loss while maintaining security. KShieldVPN will focus on efficient data routing and server load balancing to provide users with fast and stable connections without compromising encryption strength.

6. Bypass Geo-Restrictions and Censorship – Enable users to access restricted websites, streaming platforms, and services that may be blocked due to regional limitations or government censorship. By allowing users to connect to servers in different locations, KShieldVPN provides unrestricted access to global content while maintaining security and privacy.

1.4 Purpose, Scope and Applicability

Purpose:

The primary purpose of a Virtual Private Network (VPN) is to provide a secure and private connection over the internet by encrypting data and masking the user's IP address. VPNs protect sensitive information from hackers, surveillance agencies, and internet service providers (ISPs), ensuring that online activities remain confidential. They are widely used for secure remote access, allowing individuals and businesses to connect to private networks safely, especially when using public Wi-Fi. Additionally, VPNs enable users to bypass geo-restrictions and censorship, granting access to restricted content and websites. By enhancing data security, online anonymity, and unrestricted internet access, VPNs serve as essential tools for privacy-conscious individuals and organizations.

Scope:

The scope of a VPN extends well beyond merely connecting to a remote server; it encompasses a wide range of applications in both personal and corporate environments. For individual users, a VPN serves as a critical tool for enhancing online privacy, securing data transmission on public Wi-Fi networks, and bypassing geo-restrictions or censorship. Additionally, VPN technology is instrumental in protecting against cyber threats such as interception, surveillance, and unauthorized data access. By integrating robust encryption protocols and security measures, VPNs support a broad spectrum of users—from privacy-conscious individuals to large enterprises—ensuring secure, reliable, and versatile connectivity across the digital landscape.

Applicability:

A Virtual Private Network (VPN) is a versatile tool applicable in personal, corporate, and security contexts. For individuals, VPNs enhance privacy and security by encrypting data and masking IP addresses, particularly on public Wi-Fi networks. They protect sensitive information during online transactions and prevent tracking. Businesses utilize VPNs for secure remote access, allowing employees to connect to corporate networks safely, which supports productivity and flexible work arrangements. VPNs also enable users to bypass geo-restrictions and censorship, granting access to streaming services and restricted websites.

Chapter 2

Survey of Technologies

This chapter presents the existing technologies present in the industry and which all technologies are implemented in the project.

2.1 Technologies implemented for the project

1. Frontend/UI Development:

Guna2 UI is a modern UI framework for Windows Forms applications, offering an extensive range of customizable controls and enhanced design features. It provides developers with pre-built animations, gradients, and shadow effects to create visually appealing user interfaces. The framework simplifies UI development by reducing the need for extensive coding while improving application responsiveness. Guna2 UI is widely used for building sleek, modern, and interactive desktop applications.

2. Backend Development:

C# is a high-level, object-oriented programming language developed by Microsoft, designed for building scalable and secure applications. It runs on the .NET Framework, which provides a vast library of pre-built functions for networking, database management, and security. C# supports features such as automatic memory management, strong typing, and multithreading, making it efficient for large-scale applications. It is widely used in desktop, web, cloud, and enterprise software development.

3. Database Management:

Microsoft SQL Server Management Studio (SSMS) – SSMS is a database management tool developed by Microsoft for working with SQL Server databases. It provides a graphical interface for database administration tasks such as query execution, performance monitoring, and security management. SSMS allows developers to create, modify, and manage databases efficiently using Transact-SQL (T-SQL) commands. It is widely used by database administrators and developers for maintaining large-scale relational databases.

4. **Cloud Services & Deployment:**

AWS is a cloud computing platform that provides on-demand computing resources, storage, and networking services. It enables businesses to deploy, manage, and scale applications without needing physical infrastructure. AWS offers a wide range of services, including virtual servers (EC2), storage solutions (S3), and database management (RDS). Due to its high availability, security, and flexibility, AWS is a preferred choice for cloud-based solutions.

5. **Version Control & Collaboration:**

Git is a distributed version control system used for tracking changes in source code and managing collaborative development. It enables developers to create branches, merge code, and revert to previous versions, ensuring smooth workflow and project organization. Git operates through repositories, either locally or on platforms like GitHub, GitLab, and Bitbucket, allowing seamless team collaboration. Its ability to handle large-scale software projects efficiently makes it an industry-standard tool in modern development.

Chapter 3

Requirements and Analysis

This chapter presents an explanation about the problem definition, its requirements specification, which SDLC model is implemented and the project duration, your hardware and software requirements, its preliminary product description and its conceptual models

3.1 Problem definition

Without the protection of a Virtual Private Network (VPN), users are exposed to significant privacy vulnerabilities while navigating the internet. Internet Service Providers (ISPs) and other third-party entities can monitor, log, and analyse online activities, including websites visited and data transmitted. This absence of encryption leaves sensitive information—such as login credentials, financial details, and personal communications—susceptible to interception by cybercriminals. Furthermore, public Wi-Fi networks, commonly used in cafes, airports, and hotels, are inherently insecure, heightening the risk of data theft and unauthorized surveillance.

Another pressing concern is the prevalence of geographic restrictions and censorship, which limit access to content based on a user's location. Many online platforms, streaming services, and websites employ geo-blocking technologies to restrict access to content available in specific regions. This creates frustration for individuals who travel frequently or seek access to global resources, news, or entertainment. In countries with stringent internet censorship laws, citizens may find themselves unable to access vital information or engage in open communication without facing potential consequences, further underscoring the need for secure solutions.

For businesses, the absence of a reliable VPN introduces operational inefficiencies and security risks. Remote employees connecting to corporate servers over unsecured networks expose sensitive company data to potential breaches and cyberattacks. Such vulnerabilities not only jeopardize confidential information but also increase the likelihood of financial losses and reputational harm. Additionally, organizations with international operations often encounter challenges in maintaining seamless communication and secure data sharing due to varying regional internet regulations. A robust VPN solution addresses these concerns by providing encrypted, private, and unrestricted internet access, ensuring both individual and organizational security.

3.2 Requirements specification

The Software Requirement Specification (SRS) document for KShieldVPN outlines the detailed requirements and specifications necessary for its development.

3.2.1.A Functional Requirements

This section describes the functional requirements of KShieldVPN, specifying the core functionalities and user interactions within the system.

3.2.1.A.1 User Registration and Authentication:

- Users should be able to register using their email ID and password.
- Secure login and authentication mechanisms should be implemented to protect user credentials.
- Users should be able to reset their passwords in case of a forgotten or compromised account.

3.2.1.A.2 VPN Connection Management:

- Users should be able to connect to and disconnect from the VPN using a simple interface.
- The system should support multiple server locations for secure and flexible connections.
- The VPN should automatically reconnect in case of an unexpected disconnection.
- A kill switch should be implemented to block internet access if the VPN connection drops.

3.2.1.A.3 Security and Privacy Features:

- The VPN should encrypt user data using secure encryption protocols.
- IP masking should be enabled to hide the user's real IP address from websites and ISPs.
- DNS and WebRTC leak protection should be implemented to prevent accidental exposure of user information.
- No user activity logs should be stored to ensure privacy compliance.

3.2.1.A.4 User Session Management:

- Users should be able to log out at any time and terminate active VPN sessions.

- Upon reopening the application, users should see the home page if logged in, or the login page otherwise.
- The system should allow auto-login functionality for convenience while maintaining security protocols.
- Active session tracking should be available, allowing users to view and manage active connections.

3.2.1.B Non-Functional Requirements

This section outlines the non-functional requirements of KShieldVPN, specifying performance, security, usability, compatibility, maintainability, and legal considerations for the system.

3.2.1.B.1 Performance Requirements:

- The VPN should provide fast connection speeds with minimal latency during encryption and data transmission.
- The system should support multiple concurrent users without performance degradation.
- VPN server switching should be seamless, ensuring minimal downtime during location changes.
- The application should have low resource consumption, optimizing CPU and memory usage for smooth operation.

3.2.1.B.2 Security Requirements:

- The VPN should use strong encryption protocols (e.g., AES-256) to protect user data.
- Secure authentication mechanisms, including multi-factor authentication (MFA), should be implemented to prevent unauthorized access.
- The system should protect against IP leaks, DNS leaks, and WebRTC leaks to maintain user anonymity.
- Measures should be taken to prevent security threats such as man-in-the-middle (MITM) attacks and packet sniffing.

3.2.1.B.3 Usability Requirements:

- The user interface should be intuitive and easy to navigate, even for users with minimal technical knowledge.
- A simple one-click connect/disconnect feature should be available for ease of use.
- The application should provide clear error messages and troubleshooting guides for connection failures.
- The system should offer multi-language support to cater to a broader user base.

3.2.1.B.4 Compatibility Requirements:

- The VPN should be compatible with Windows operating systems, ensuring smooth integration with different versions.
- It should support multiple network types, including Wi-Fi, mobile data, and Ethernet connections.
- The system should be optimized for different screen resolutions and device configurations.
- Performance testing should be conducted to validate stability under various network conditions.

3.2.1.B.5 Maintainability Requirements:

- The codebase should be well-structured, modular, and documented to facilitate future maintenance and enhancements.
- Developers should be able to easily understand and modify the system, with clear separation of concerns and reusable components.
- The system should allow for seamless updates, ensuring security patches and new features can be deployed without major disruptions.
- Logs and error reports should be systematically recorded to aid debugging and performance analysis.

3.2.1.B.6 Legal and Compliance Requirements:

- The VPN should comply with data protection and privacy laws, such as GDPR, to ensure user data is handled securely.
- The system should adhere to no-log policies, ensuring that user activity is not stored or tracked.
- Encryption standards should meet industry regulations for secure data transmission.
- The software should comply with export control laws and country-specific VPN regulations to avoid legal conflicts.

3.3 SDLC

Agile Model

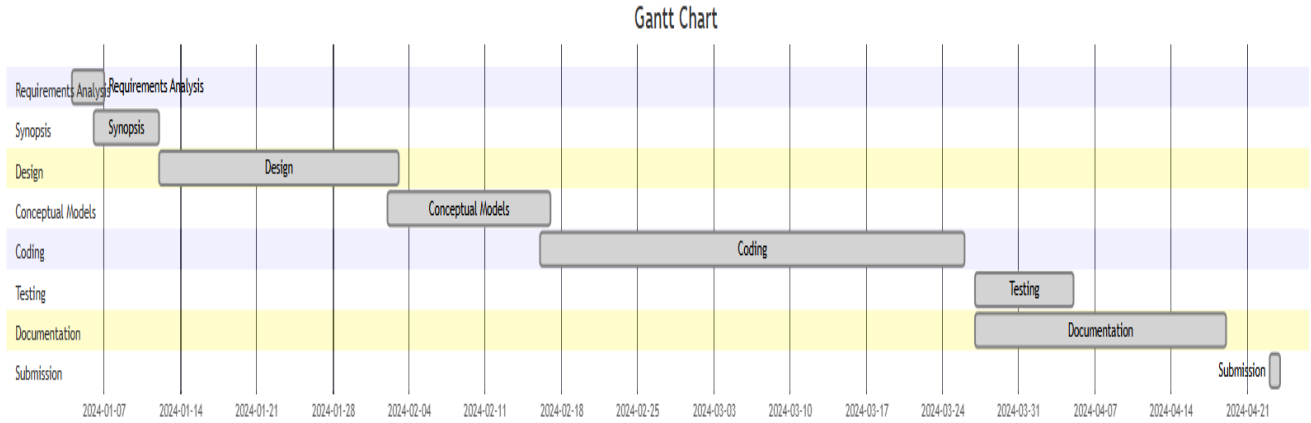
The Agile model is an iterative and incremental approach to software development that emphasizes flexibility, collaboration, and customer satisfaction. It involves continuous stakeholder engagement, regular testing and validation, and the ability to adapt to changing requirements throughout the project.

Why Agile is Suitable:

- Evolving Security Requirements: The cybersecurity landscape is dynamic, with frequent changes in threats, protocols, and privacy regulations. Agile allows the system to adapt quickly to new security standards and technologies.
- User Feedback Integration: Agile encourages regular interaction with users. Feedback from VPN users regarding usability, performance, and features like server selection or connection reliability can be incorporated in each iteration.
- Frequent Testing for Stability: Agile emphasizes continuous testing and integration, which is crucial for VPN systems to ensure secure connections, proper encryption, DNS leak protection, and overall stability.
- Risk Management: Through its iterative nature, Agile helps identify and resolve issues early—whether it's in authentication, session handling, or payment processing—thus reducing the chance of critical failures in production.
- Feature Prioritization and Delivery: Features like user login, OpenVPN connection, server switching, speed test, DNS leak test, password recovery, and Stripe payment integration can be developed and delivered incrementally based on priority and user demand.
- User-Centric Design: Agile ensures that the VPN remains intuitive, secure, and user-friendly by involving end-users throughout the development process. This is vital for a product like KShieldVPN, where simplicity and trust are key.

Figure 3.3 Agile Model

3.4 Planning and scheduling



Description of the phases included within the Gantt Chart:

1. Requirements Analysis

Here, the topic for the project has been approved by the guide and the requirements such as the technologies, functionalities and more are collected to prepare the synopsis.

Commenced on January 4, 2024

Concluded on January 6 2024

2. Synopsis

Here, the preparation of the initial synopsis takes place which involves a brief explaining the project in short.

Commenced on January 6 , 2024

Concluded on January 11, 2024 (Submitted)

3. Design

Here, the application's architecture and user interfaces were developed, keeping in mind the flow of the application between modules.

Commenced on January 12, 2024

Concluded on February 2, 2024

4. Conceptual Models

In this phase, the conceptual models were developed such as the Use-Case diagram, Class diagram etc., to guide the coding process.

Commenced on February 2, 2024

Concluded on February 16, 2024

5. Coding

This phase is where the actual coding phase of the application takes place. It is the actual development of the application's front-end and back-end components

Commenced on February 16, 2024

Concluded on March 25, 2024

6. Testing

The testing phase included all the different tests that were carried out in order to ensure that the application met all the specifications and functioned as expected.

Commenced on March 27, 2024

Concluded on April 4, 2024

7. Documentation

This phase includes the timeline that the project report and documentation was prepared.

Commenced on March 27, 2024

Concluded on April 18, 2024

8. Submission

The project concluded with the final submission on April 23, 2024

3.5 Software and Hardware Requirements for KShieldVPN

3.5.1 Client-Side Software Requirements

Sr. No	Name	Specification
i.	Operating System	Windows 10 or later
ii.	VPN Client	OpenVPN
iii.	.NET Framework	.NET Framework 4.8+
iv.	Internet Connection	Stable Wi-Fi or Ethernet

3.5.2 Client-Side Hardware Requirements

Sr. No	Name	Specification
i.	Processor	Intel Core i3 or AMD equivalent
ii.	RAM	4GB minimum (8GB recommended)
iii.	Storage	At least 500MB free space for installation
iv.	Monitor	14” Color Monitor or higher
v.	Network Adapter	Ethernet or Wi-Fi adapter for internet access

3.5.4 Developer-Side Software Requirements

Sr. No	Name	Specification
i.	Operating System	Windows 10 or later
ii.	VPN Software	OpenVPN
iii.	Framework	.NET Framework 4.8+
iv.	UI Library	Guna2 UI for WinForms
v.	Code Editor	Visual Studio 2019/2022
vi.	Database	Microsoft SQL Server Management Studio (SSMS)
vii.	Cloud Services	AWS
viii.	Version Control	Git
ix.	Internet Connection	Stable Wi-Fi or Ethernet

3.5.5 Developer-Side Hardware Requirements

Sr. No	Name	Specification
i.	Processor	Intel Core i5 or AMD equivalent (recommended)
ii.	RAM	8GB minimum (16GB recommended)
iii.	Storage	SSD (At least 10GB free space)
iv.	Monitor	14” Color Monitor or higher
v.	Network Adapter	Ethernet or Wi-Fi adapter for internet access

3.6 Preliminary product description

The Preliminary Product Description section of the Requirements and Analysis in a project report provides an overview of the product, its features, intended use, and the problem it seeks to address. This description establishes the foundation for understanding how the product will meet user needs and align with project goals.

The proposed KShieldVPN is a basic yet secure Virtual Private Network (VPN) solution designed to ensure user privacy, protect sensitive data, and provide secure internet access. It is tailored for individuals who prioritize safeguarding their online activity from cyber threats, masking their digital footprint, and bypassing geo-restrictions or network-level censorship. By offering a simple, efficient, and reliable solution, KShieldVPN empowers users with control over their internet experience while maintaining a focus on security and usability.

At its core, KShieldVPN features a clean and user-friendly interface built using the Guna2 UI framework. The application is divided into two primary sections: a connection panel and a status/reporting panel. The connection panel serves as the central hub for managing VPN sessions. Users can select a server profile, authenticate their credentials, and initiate or terminate the secure connection with minimal effort.

The status panel provides real-time updates on the connection status, including details such as the user's current IP address, session duration, and basic logs of network activity. Additionally, the application includes integrated tools such as a speed test utility, which allows users to measure the performance of their VPN connection. A DNS leak test tool is also included, enabling users to verify that DNS queries are being routed securely through the encrypted tunnel. These tools enhance transparency and allow users to monitor both the efficiency and security of their connection.

Underpinning the functionality of KShieldVPN is the OpenVPN protocol, a widely trusted and open-source encryption standard known for its robust security capabilities. Upon launching the application, users can securely connect to a predefined VPN server by entering their authentication credentials and initiating the encrypted tunnel. This ensures that all internet traffic is routed through the secure connection, shielding users from eavesdropping, man-in-the-middle attacks, and other malicious activities.

To further enhance usability, KShieldVPN incorporates features such as IP address masking, which hides the user’s real IP address to prevent tracking. The system is also designed to log minimal session data, adhering to basic privacy standards to ensure that user activity remains confidential. While this version of KShieldVPN does not include advanced features such as dynamic server switching, multi-platform support, or custom server configurations, it provides a stable and secure connection suitable for basic privacy needs.

The inclusion of tools like the speed test and DNS leak test ensures that users can verify both the performance and integrity of their connection. These features add an extra layer of confidence in the application's functionality, making it a reliable choice for secure browsing.

KShieldVPN is particularly well-suited for users seeking a lightweight, demo-level VPN solution for educational, testing, or introductory purposes. Its simplicity makes it an ideal platform for individuals who want to learn about VPN technology, experiment with secure browsing, or evaluate the essential components of a functional VPN client.

In its current form, KShieldVPN strikes a balance between functionality and accessibility, offering a practical and reliable solution for users who require a straightforward yet effective means of protecting their online presence. By focusing on usability, transparency, and security, KShieldVPN lays the groundwork for continued development and expansion, positioning itself as a valuable resource for both novice and intermediate users alike.

3.7 Conceptual models

3.7.1 Use Case Diagram

A use case diagram visually illustrates the ways in which users, or actors, interact with a system. It outlines the connections between these actors and the various use cases, or actions, that they participate in. These diagrams are commonly utilized in software design to offer a clear depiction of the system's planned functionalities and how users will engage with them.

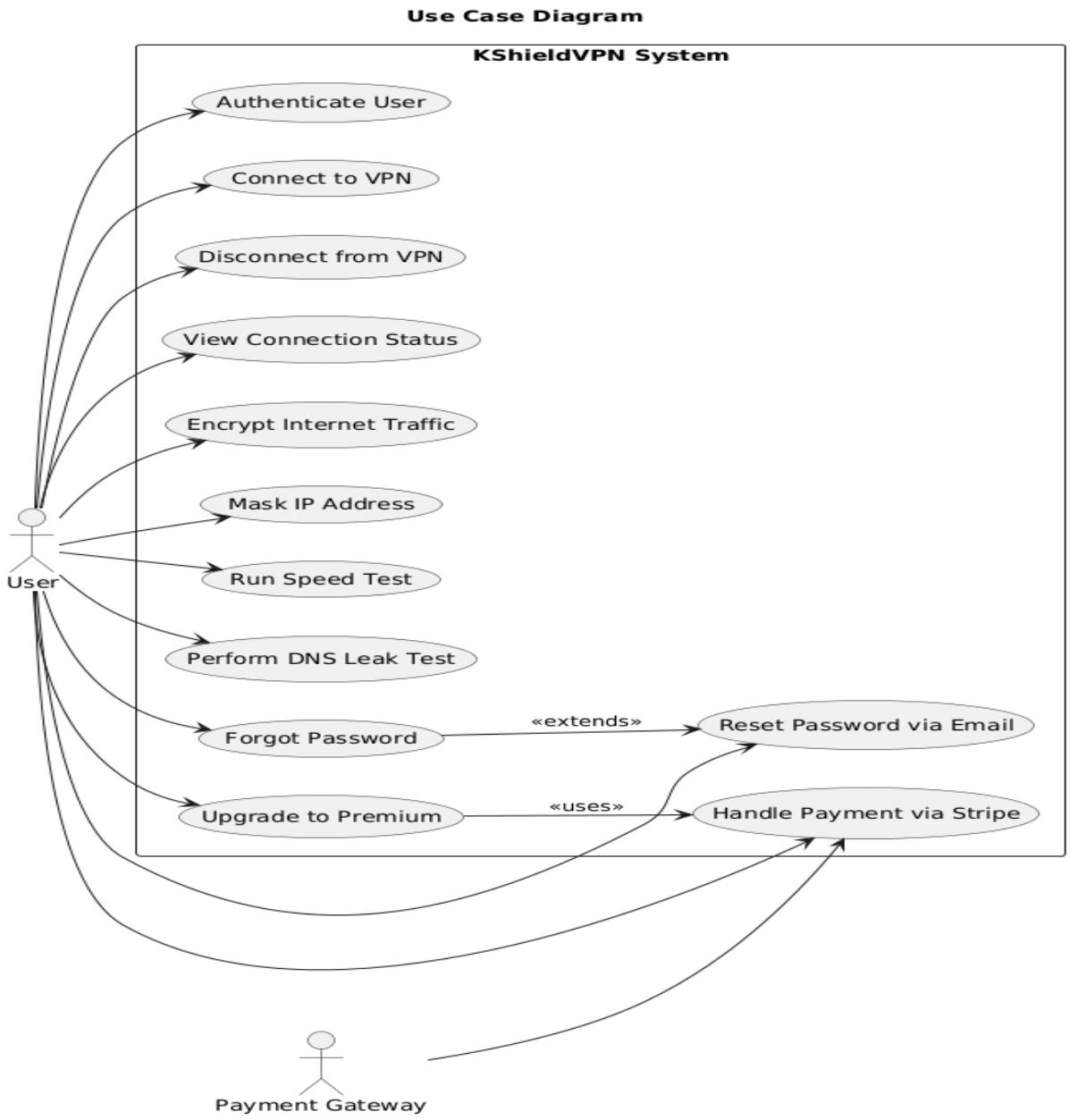


Figure 3.7.1 Use Case Diagram

3.7.2 DFD Level Diagram

A DFD is a visual depiction of how data moves through an information system, focusing on its processing aspects. It illustrates the way input data is converted into output results by passing through a series of functional transformations. Typically, DFDs are used to represent and analyse the flow of data processing within a system. They also help in identifying and organizing the various components of a system, making it easier to understand and design complex processes.

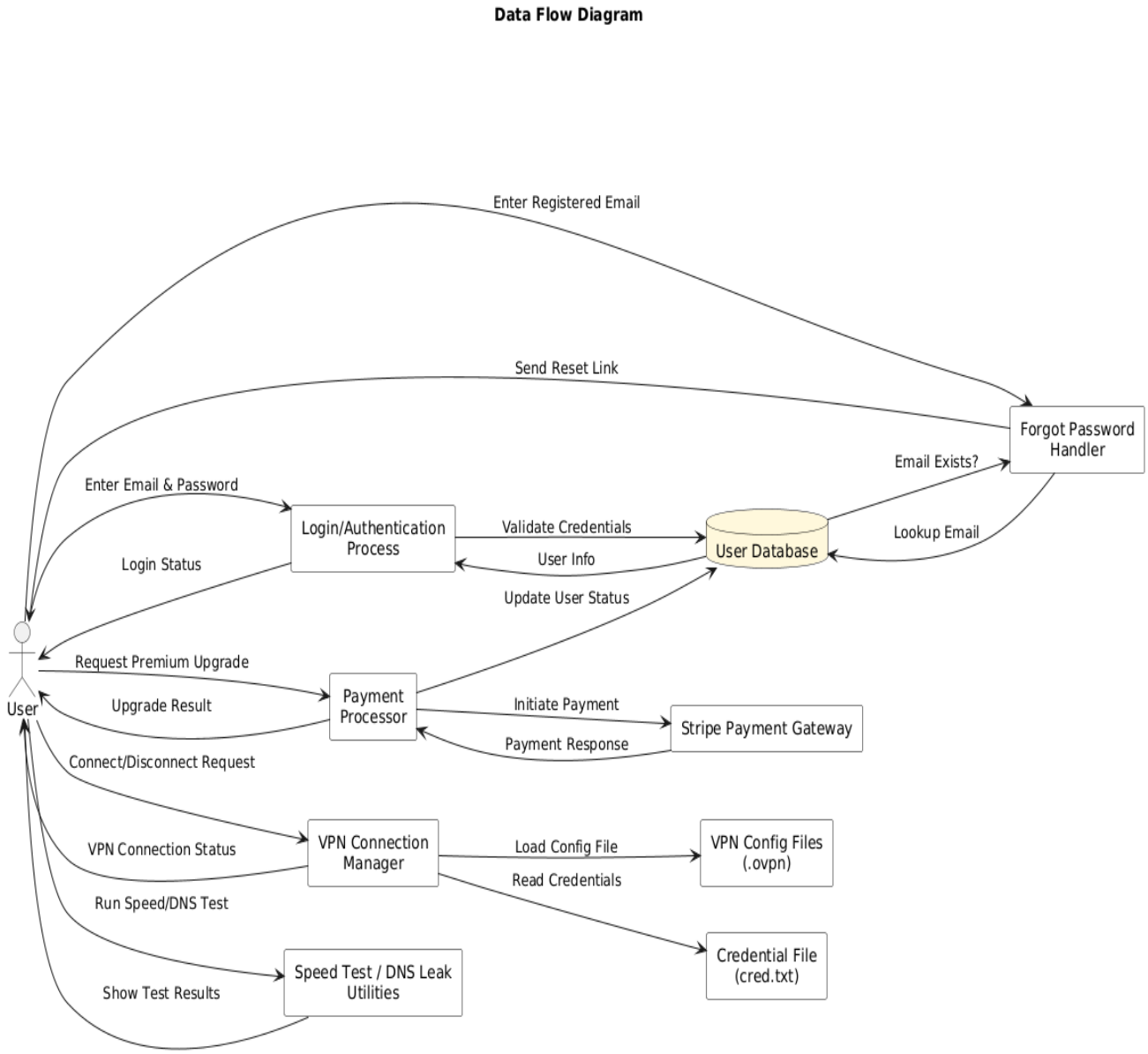


Figure 3.7.2 DFD Level Diagram

3.7.3 ER Diagram

An ER diagram is a data model that illustrates the connections between entities, such as objects or concepts, within a system. It is commonly utilized in the design of databases. The key elements include entities, which are depicted as rectangles, relationships shown as diamonds, and attributes represented by ovals.

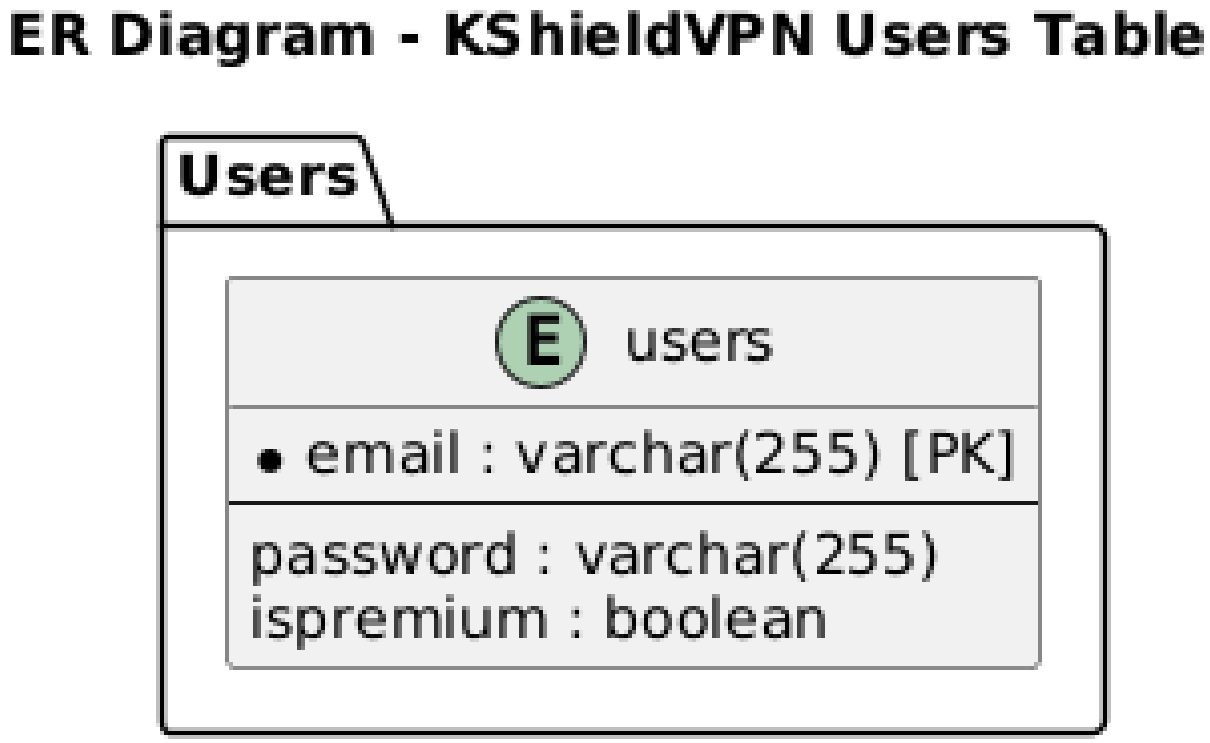


Figure 3.7.3 ER Diagram

3.7.4 Class diagram

A class diagram is a static structure diagram in the Unified Modelling Language (UML) that depicts the organization of a system by displaying its classes, their attributes, and the connections between them. It is extensively utilized in object-oriented modelling and software development to represent the system's design.

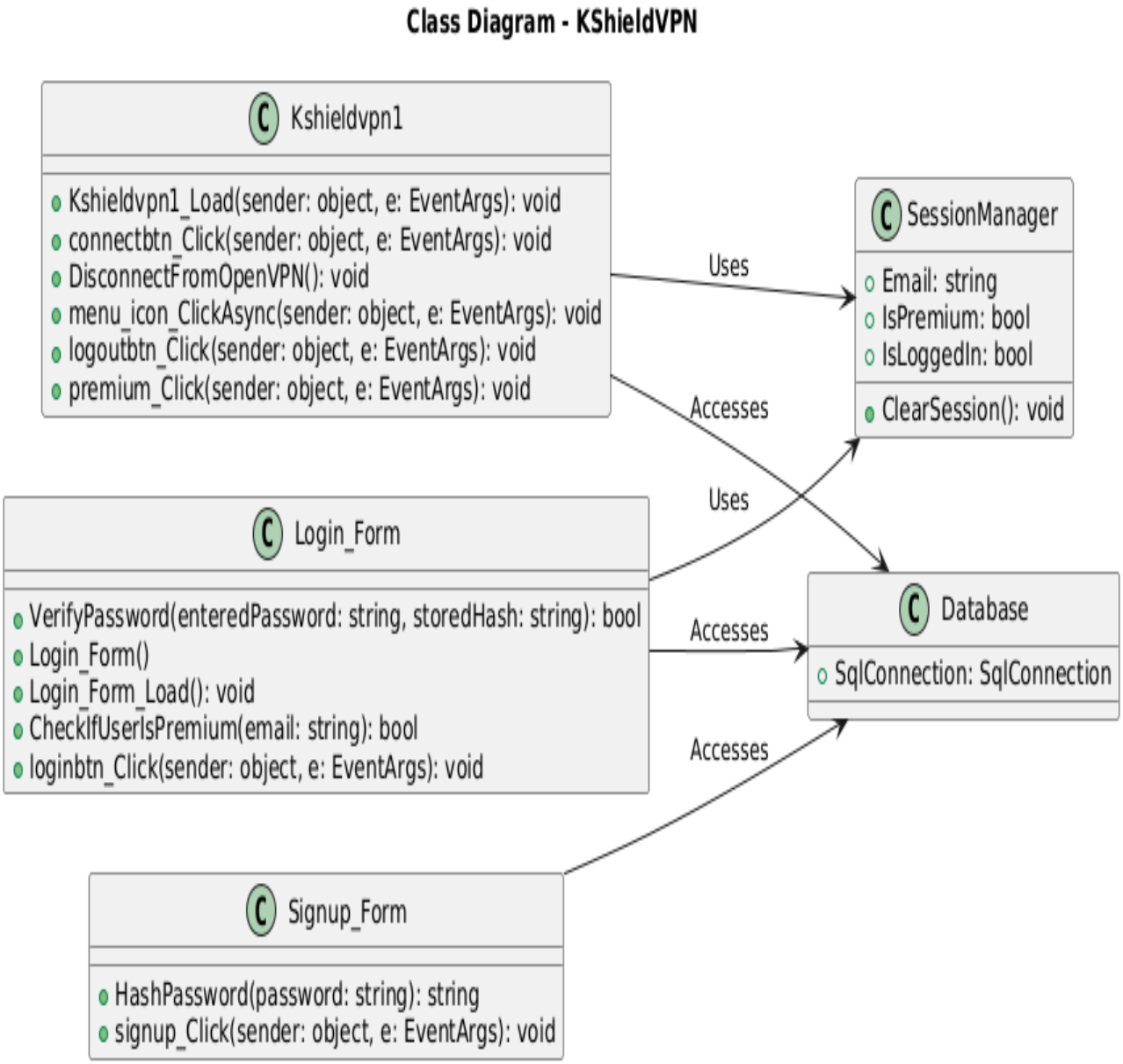


Figure 3.7.4 Class Diagram

3.7.5 Activity Diagram

An activity diagram is a flowchart-like representation that models' workflows or processes, illustrating the progression of activities from the beginning to the end. Activities, depicted as rounded rectangles, are linked by arrows to show the flow between them.

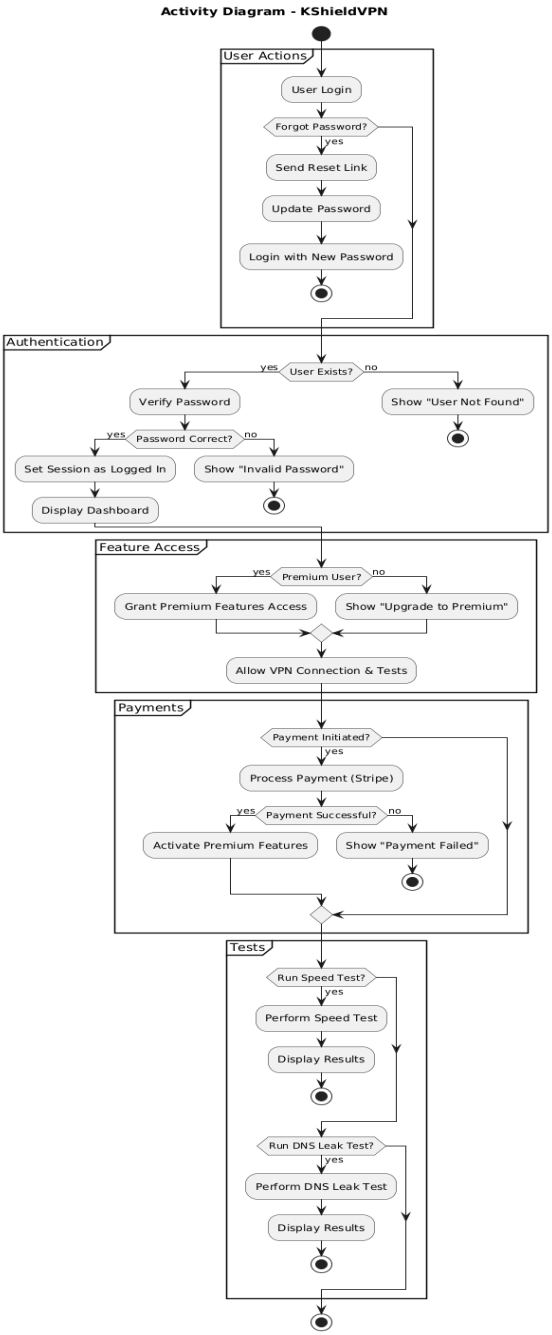
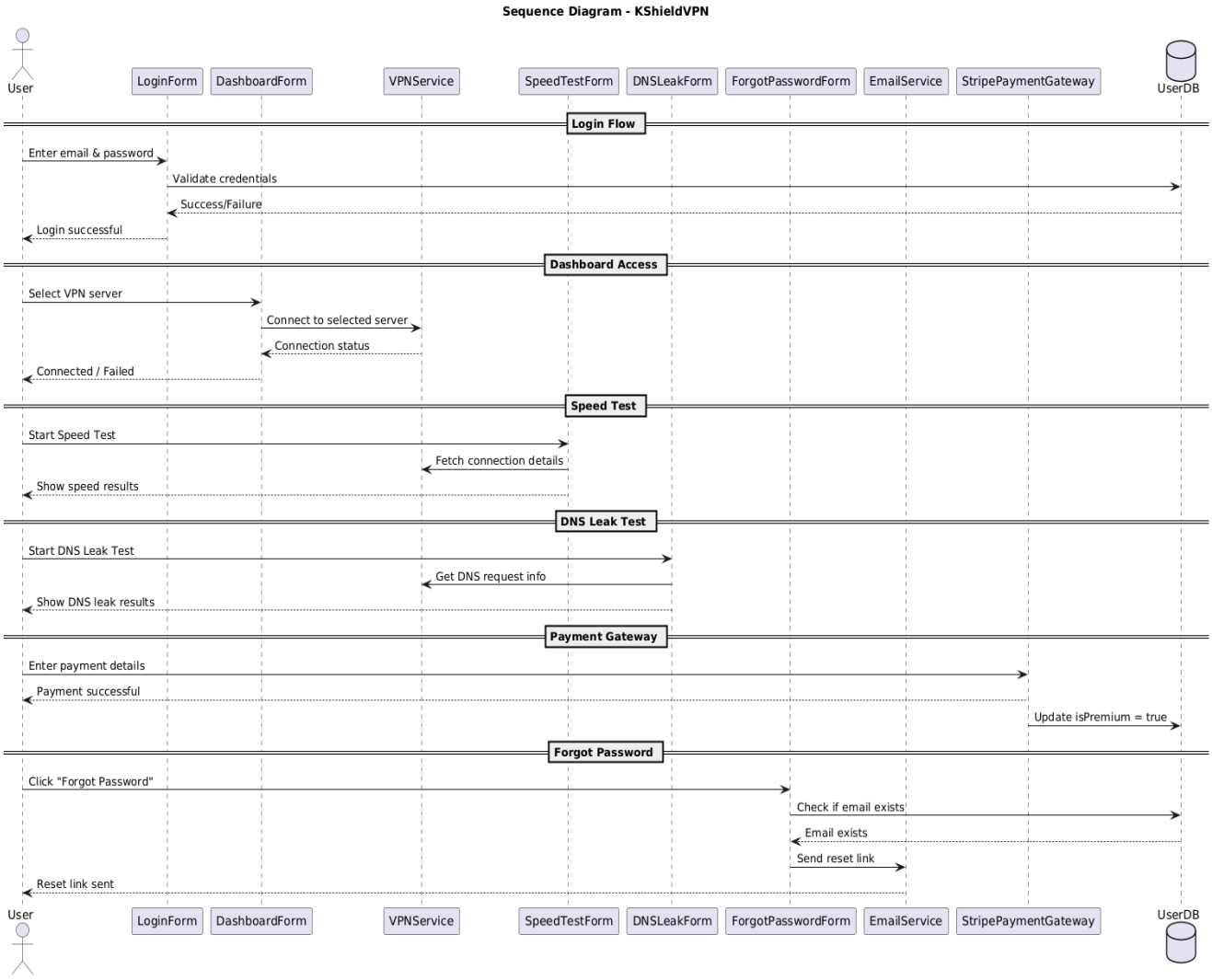


Figure 3.7.5 Activity Diagram

3.7.6 Sequence diagram

A Sequence Diagram is a key component of used to visualize the interaction between objects in a sequential order. It focuses on how objects communicate with each other over time, making it an essential tool for modeling dynamic behavior in a system. Sequence diagrams illustrate object interactions, message flows, and the sequence of operations, making them valuable for understanding use cases, designing system architecture, and documenting complex processes.



3.7.6 Sequence Diagram

Chapter 4

System Design

This chapter presents about the basic modules used, its data design, procedural design, user interface design, security issues and test cases design.

4.1 Basic Modules

The KShield VPN is a highly intuitive and user-friendly Windows application specifically designed to offer secure and seamless VPN connectivity to its users. The application has been meticulously crafted to ensure that even individuals with minimal technical expertise can easily navigate and utilize its features. Below is an in-depth breakdown of the key modules that make up the KShield VPN application:

1. User Authentication Module

The User Authentication Module is a cornerstone of the KShield VPN application, ensuring that user accounts are securely managed and protected. This module handles both the registration and login processes, providing a robust framework for user access.

Key Features Include:

- Signup Page:

  New users are guided through a simple and secure account creation process. They are required to enter essential credentials such as their email address and password. To safeguard user data, the system employs advanced hashing algorithms to store passwords securely, ensuring that sensitive information remains protected from unauthorized access.

- Login Page:
  Existing users can access their accounts by entering their registered credentials. The login process is streamlined to minimize friction while maintaining high security standards. Once authenticated, users are granted access to the application's full range of features.

- Session Management:

  The application implements session handling to enhance user convenience. Logged-in users are automatically directed to the home page, where they can begin interacting with the application. If a user logs out, they are redirected to the login screen, ensuring that unauthorized access is prevented.

2. Home Page Module

The Home Page Module serves as the central hub of the KShield VPN application, offering users a unified dashboard to access all available features. This module is designed to provide a seamless and efficient user experience, allowing users to manage their VPN connections and explore additional tools with ease.

Key Features Include:

- Server Selection:

  Users have the flexibility to choose from a variety of VPN server locations, including popular options like Seoul, Paris, and other global destinations. This feature ensures that users can establish secure connections tailored to their specific needs, whether for privacy, performance, or accessing region-restricted content.

- Connection Control:

  The application simplifies the process of connecting to or disconnecting from the VPN. With just a single click, users can initiate a secure connection or terminate it as needed. This intuitive control mechanism makes it easy for users to manage their online activities without any hassle.

- Navigation:
  The Home Page provides quick and easy access to other essential tools within the application. Users can seamlessly navigate to features such as the speed test, DNS leak test, and payment page, ensuring that all necessary functionalities are readily available at their fingertips.

3. Speed Test Module

The Speed Test Module is a valuable tool designed to help users evaluate the performance of their internet connection while using the KShield VPN. This feature ensures that users can verify the quality and reliability of their VPN service.

Key Features Include:

- Speed Metrics:

  Users can measure critical aspects of their internet connection, such as download and upload speeds. These metrics provide valuable insights into the overall performance of the VPN, helping users determine whether the selected server meets their expectations.

- Result Display:

  The application presents the test results in a visually engaging format, making it easier for users to interpret the data. Clear and concise visualizations ensure that users can quickly assess their connection's performance and make informed decisions about server selection.

4. DNS Leak Test Module

The DNS Leak Test Module is a crucial component of the KShield VPN application, designed to safeguard user privacy by verifying that DNS queries remain within the secure VPN tunnel. This feature ensures that users' online activities remain anonymous and protected from potential leaks.

Key Features Include:

- Leak Detection:

  The application performs a thorough check to determine whether DNS requests are being routed through the VPN. This proactive approach helps identify any vulnerabilities that could compromise user privacy.

- Privacy Assurance:

  If a DNS leak is detected, the application immediately notifies the user, providing clear instructions on how to address the issue. This ensures that users can take corrective actions to maintain full anonymity and protect their online presence.

5. Payment Gateway Module

The Payment Gateway Module is an integral part of the KShield VPN application, enabling users to unlock premium features and access enhanced functionality through a secure payment system. This module ensures that transactions are processed smoothly and securely.

Key Features Include:

- Payment Integration:

  Users can complete payments using a connected demo payment gateway, such as Stripe. The integration is designed to be seamless, providing a hassle-free experience for users who wish to upgrade to premium services.

- Success Handling:

  Upon successful completion of a payment, users are granted immediate access to premium VPN servers and exclusive features. This includes faster connection speeds, additional server locations, and advanced security settings, enhancing the overall user experience.

Each module within the KShield VPN application has been thoughtfully designed to enhance user experience, ensure maximum security, and provide smooth and reliable VPN connectivity. The combination of these modules creates a cohesive and structured interface that caters to both novice and experienced users. Whether signing up for the first time, managing a secure connection, testing network performance, or upgrading to premium features, the KShield VPN application offers a comprehensive solution that prioritizes usability, privacy, and performance. By leveraging these modules, users can confidently navigate the digital landscape, knowing that their online activities are protected and optimized for efficiency.

4.2 Data Design

The KShield VPN application adopts a streamlined and focused database design, meticulously crafted to align with its core functionalities: user authentication, access control, and differentiation between free and premium users. While this is a demo project, the database structure has been intentionally kept minimalistic to ensure clarity and efficiency while still supporting the essential features required for VPN access and feature management. This approach strikes a balance between simplicity and functionality, providing a robust foundation for the application's current requirements.

At the heart of the database lies the Users table , which serves as the primary repository for user-related information. This table is designed to be both lightweight and secure, ensuring that critical operations such as user verification and access control can be executed quickly and reliably. The Users table consists of three key fields, each playing a vital role in the application's functionality:

1. Email:
   The Email field acts as the unique identifier for each user within the system. It serves dual purposes: first, it provides a reliable means of referencing user accounts during login and other account-related operations; second, it ensures that each user is uniquely identifiable within the database. By using email addresses as unique identifiers, the application leverages a widely adopted standard for user identification, making the process intuitive and familiar for users. Additionally, the uniqueness constraint on this field prevents duplicate accounts from being created, further enhancing data integrity.

2. Password Hash:

   Security is a top priority in the KShield VPN application, and the Password Hash field exemplifies this commitment. Instead of storing passwords in plaintext—a practice that poses significant risks in the event of a data breach—the application employs the SHA-256 hashing algorithm to securely store password hashes. This cryptographic technique transforms user passwords into irreversible hash values, ensuring that even if the database is compromised, the actual passwords remain protected. The use of SHA-256 provides a strong layer of security, safeguarding user credentials against unauthorized access. During the login process, the system hashes the entered password and compares it with the stored hash, verifying the user's identity without ever exposing sensitive information.

3. IsPremium:
   The IsPremium field is a boolean attribute that determines the level of access granted to each user. This field plays a pivotal role in differentiating between free and premium users. If the value is set to true , the user is recognized as a premium subscriber and gains

access to advanced features such as enhanced server selection, speed testing, DNS leak detection, and other exclusive functionalities. Conversely, if the value is false , the user is restricted to basic access, which includes limited server options and standard connectivity features. This binary flag enables the application to implement access control logic efficiently, ensuring that users are presented with the appropriate interface and features based on their subscription status.

The database design is intentionally minimalistic, focusing only on the essential fields required to support the application's core functionalities. By limiting the number of fields to just three—Email, PasswordHash, and IsPremium—the database achieves a high degree of efficiency. Queries related to user authentication and access control are optimized for speed and simplicity, as the reduced dataset size minimizes processing overhead. For example, when a user logs in, the system performs a quick look up to verify the email and hashed password, followed by a check of the IsPremium flag to determine the user's access level. This streamlined process ensures that login and session management are both fast and reliable.

Additionally, the lightweight nature of the database supports seamless session management. Once a user successfully logs in, their session is retained, allowing them to navigate the application without repeatedly entering their credentials. The system uses the IsPremium flag to dynamically adjust the user interface, displaying premium features only to eligible users. This ensures that the user experience is tailored to each individual's subscription status, enhancing both usability and satisfaction.

4.3 Procedural design

The Procedural Design section provides a comprehensive breakdown of the step-by-step operations performed by the KShieldVPN system to deliver its core functionalities. This design ensures that users experience a seamless, secure, and efficient interaction with the application while maintaining high standards of usability and performance. Below is a detailed explanation of each procedural flow, encompassing user registration, login management, server selection, VPN connectivity, session handling.

1. User Registration and Login

The user registration and login process are a critical component of the KShieldVPN application, ensuring secure access and account management for all users. The procedural steps are as follows:

i. GUI Interface for Registration and Login:

The system provides an intuitive graphical user interface (GUI) that enables new users to create accounts and existing users to log in securely. This interface is designed to be user-friendly, guiding users through the necessary steps to complete their actions without confusion.

ii. Registration Process:

During the registration process, the system validates user input to ensure that all required fields, such as email and password, are correctly filled out. Once validated, the user's password is hashed using the SHA-256 cryptographic algorithm, ensuring that sensitive information is stored securely in the SQL Server database. The system also enforces uniqueness for email addresses to prevent duplicate accounts.

iii. Login Process:

When a user attempts to log in, the system retrieves the stored password hash from the database and compares it with the hash of the entered password. If the credentials match, the user is granted access and redirected to the dashboard, where they can begin interacting with the application. In case of unsuccessful login attempts, the system displays an appropriate error message, prompting the user to re-enter their credentials.

2. Server Selection and VPN Connection

Once logged in, users gain access to the dashboard, which serves as the central hub for managing their VPN connections. The server selection and connection process are outlined below:

i. Server Selection via Dropdown Menu:

The dashboard features a dropdown menu that allows users to select a preferred VPN server location, such as Seoul or Paris.

ii. Establishing the VPN Connection:

After selecting a server, the system invokes the OpenVPN client via the command line, passing the associated .ovpn file and user credentials as parameters. The connection process runs in the background, ensuring minimal disruption to the user experience. Upon successful establishment of the VPN tunnel, users are notified through a confirmation message. In the event of an error during the connection process, the system provides detailed feedback to help users troubleshoot the issue.

iii. Background Operations:

The system ensures that all background processes related to the VPN connection are managed efficiently. This includes monitoring the connection status and providing real-time updates to the user interface. By running these operations in the background, the application maintains a responsive and user-friendly environment.

3. VPN Disconnection

To provide users with full control over their VPN sessions, the KShieldVPN application includes a straightforward disconnection mechanism:

i. Manual Disconnection Option:

Users can terminate their VPN connection at any time by clicking the "Disconnect" button on the dashboard. This action triggers the system to forcefully terminate the `openvpn.exe` process using the `taskkill` command.

ii. Ensuring Proper Closure of the VPN Tunnel:

The disconnection process is designed to ensure that the VPN tunnel is properly closed, preventing any lingering connections or security vulnerabilities. Once the process is successfully terminated, users are notified that the disconnection has been completed.

4. Speed Test and DNS Leak Test

The KShieldVPN application includes built-in tools for testing network performance and verifying privacy protection. These features are accessible directly from the dashboard:

i. Navigation to Testing Tools:

The dashboard provides buttons that allow users to navigate to the Speed Test form and DNS Leak Test form. These forms open in separate windows, temporarily hiding the dashboard to focus the user's attention on the test results.

ii. Speed Test Functionality:

The Speed Test tool measures the user's internet connection performance by calculating key metrics such as download and upload speeds. The results are displayed in a clear and visually engaging format, enabling users to assess the impact of the VPN on their connection quality.

iii. DNS Leak Test Functionality:

The DNS Leak Test tool verifies whether DNS queries are being routed through the secure VPN tunnel. If any leaks are detected, the system notifies the user and provides guidance on how to address the issue. This ensures that users' online activities remain private and protected.

iv. Returning to the Dashboard:

After completing a test, users can easily return to the dashboard by closing the test window. This seamless navigation enhances the overall user experience, allowing users to switch between tasks without interruption.

5. Session Management and Logout

Effective session management is essential for maintaining security and usability within the KShieldVPN application. The logout process is designed to ensure that user sessions are terminated securely:

i. Logout Option on the Dashboard:

Users can log out from the dashboard at any time by clicking the "Logout" button. This action initiates the session termination process.

ii. Clearing Saved Credentials and Preferences:

During logout, the system clears the saved user email and "Remember Me" flag from the application settings. This ensures that no residual data remains accessible after the session ends.

iii. Redirecting to the Login Screen:

Upon completing the logout process, the system redirects the user to the login screen, ensuring that unauthorized access is prevented. This step reinforces the application's commitment to secure session handling.

4.4 User Interface design

The User Interface (UI) Design of the KShieldVPN system focuses on providing a seamless and intuitive experience for users through a Windows Forms application, enhanced with Guna2 UI components. The design ensures simplicity, security, and clarity in navigation and VPN usage.

1. Login and Registration

- A. The application includes a clean, modern login and registration form.
- B. Input fields for Email and Password are clearly labelled, with Login and Register buttons styled using Guna2 controls.
- C. Input validation is performed for empty or incorrect entries, with user-friendly error messages.

2. Dashboard Interface

- A. After logging in, users are directed to the main Dashboard (Kshieldvpn1).
- B. The dashboard provides:
    - A ComboBox to select server locations (e.g., *Seoul*, *Paris*).
    - Buttons for Connect, Disconnect, Speed Test, DNS Leak Test, and Logout.
- C. Informative pop-ups and MessageBox alerts are used to notify users of VPN status or errors.

3. Server Selection and VPN Control

- A. Server options are available in a dropdown
- B. On clicking Connect, the app silently executes OpenVPN with saved credentials, and a success message is displayed.
- C. The Disconnect button stops the VPN.

4. Speed Test and DNS Leak Detection

- A. Dedicated buttons launch separate forms:

- Speed Test → stform
- DNS Leak Test → dnsleakform

- B. Each form opens in a new window and temporarily hides the dashboard.
- C. After tests, users can return to the main dashboard via navigation buttons or by closing the test forms.

5. Side Menu and Navigation

- A. A collapsible side menu (menupanel) is integrated for enhanced UI control.
- B. The menu includes:
  - Navigation options for Dashboard, Tests, and Logout.
  - Smooth transition animations using guna2Transition1.
- C. Menu state (collapsed/expanded) is toggled with a button (menu_icon), providing a clean and responsive layout.

6. Logout and Session Management

- A. A visible Logout button ends the user session and redirects to the login screen.
- B. On logout, saved login data (Email) is cleared via app settings.

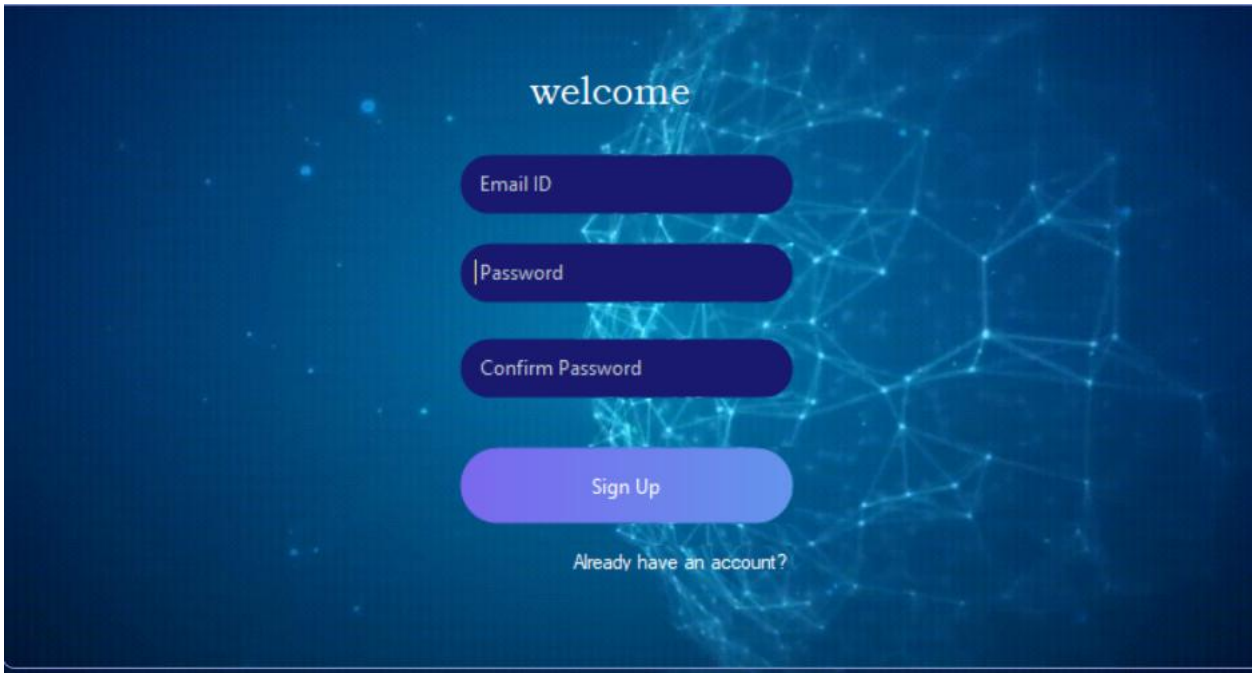


Figure 4.4.1 User Signup Design

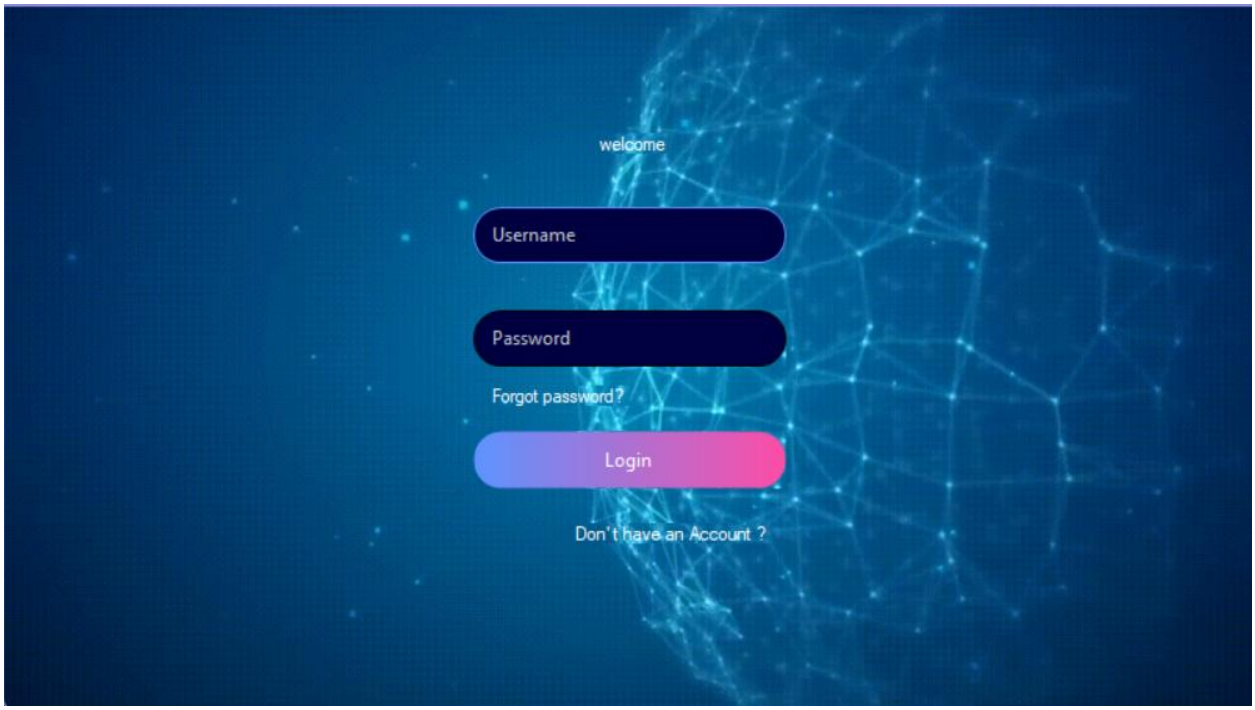


Figure 4.4.2 User Login Design

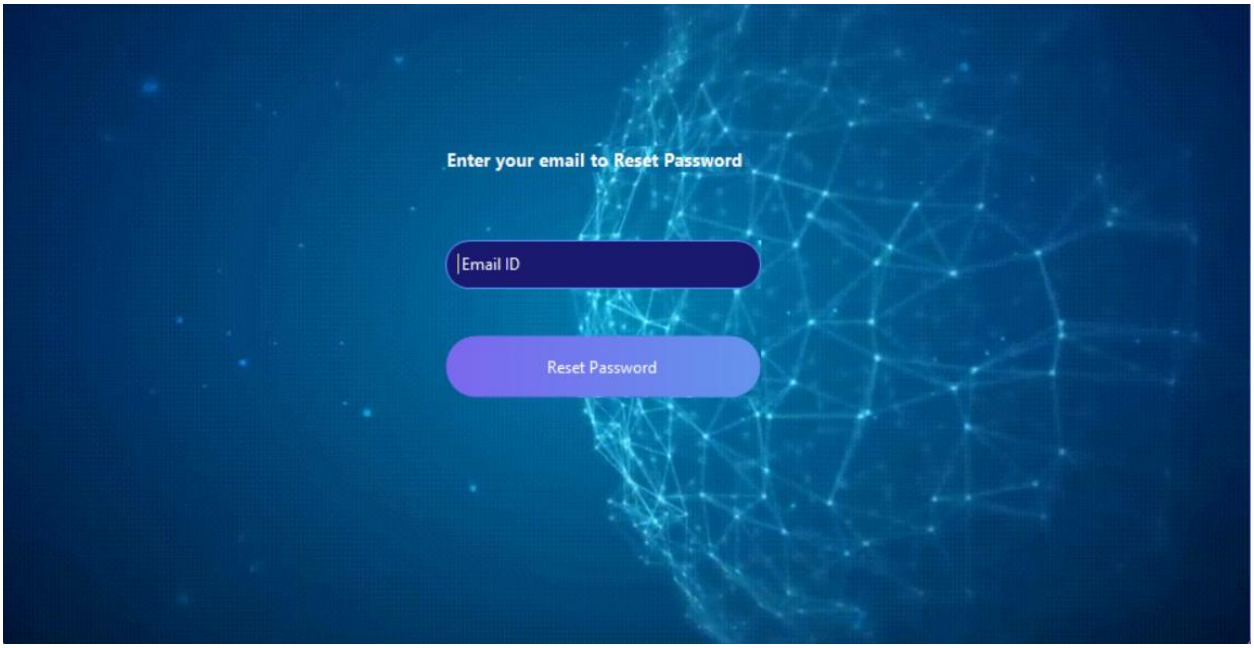


Figure 4.4.3 Forgot Password Design

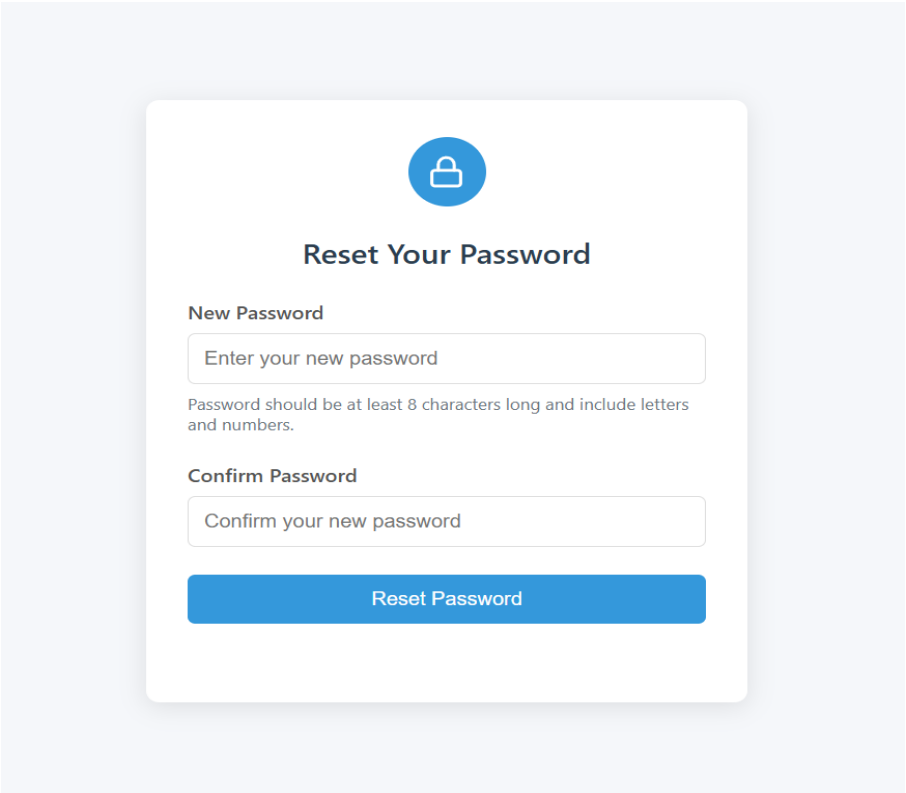


Figure 4.4.4 Reset Password Design

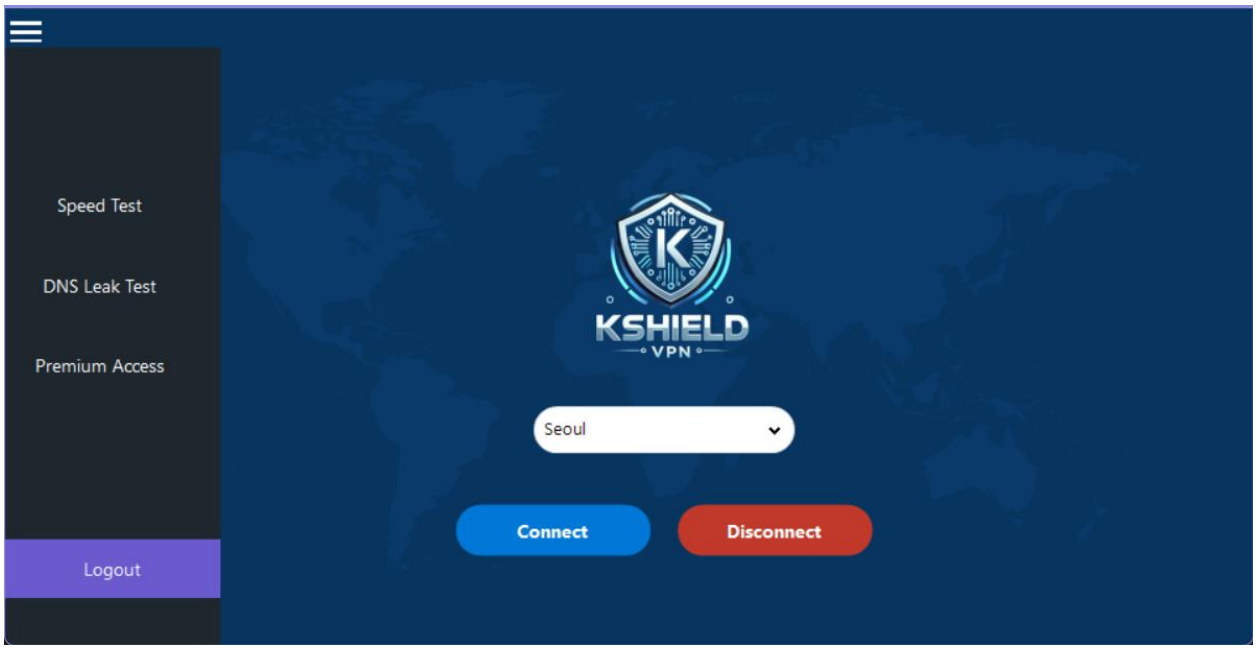


Figure 4.4.5 Home Page Design

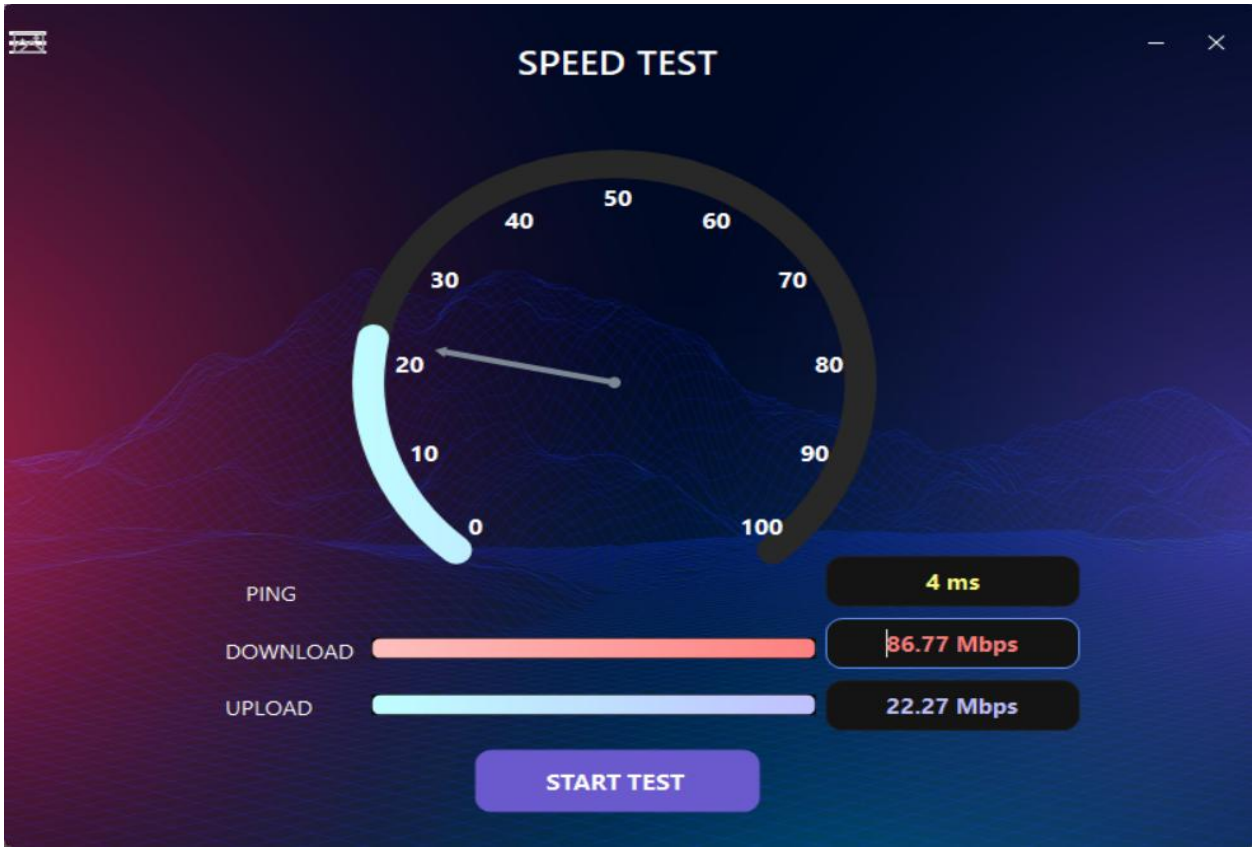


Figure 4.4.6 Speed Test Design

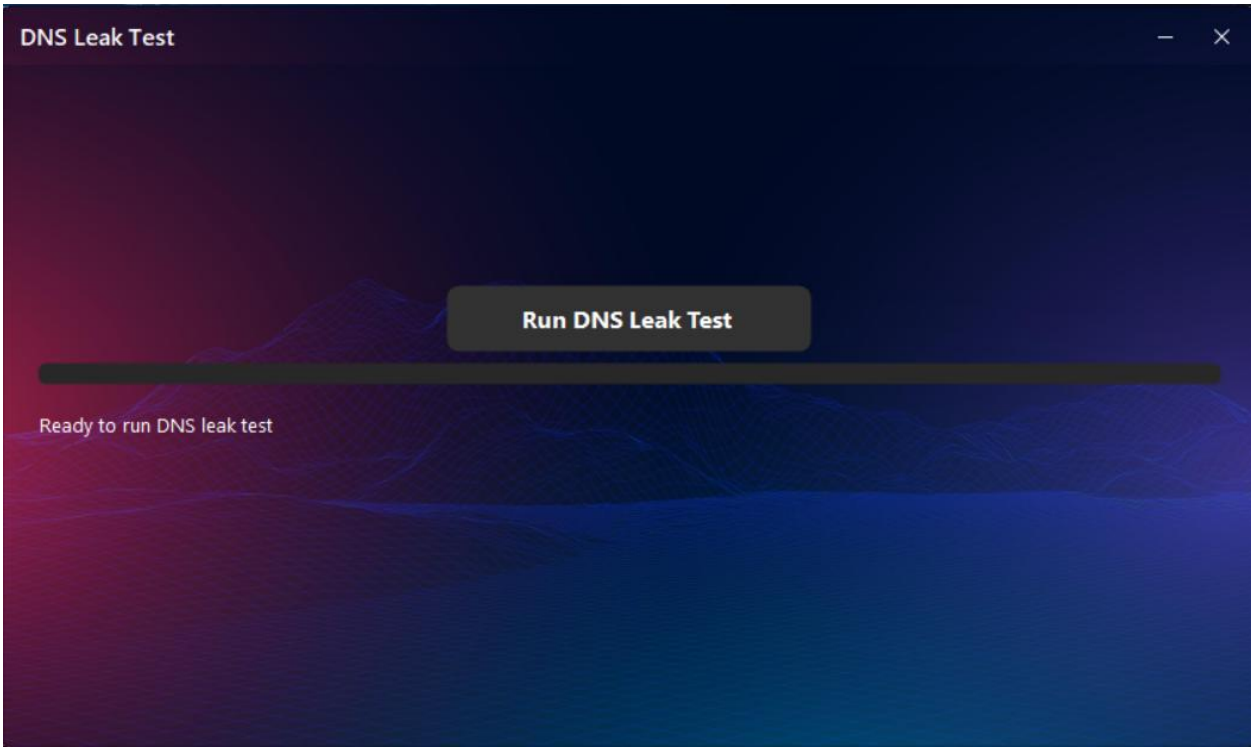


Figure 4.4.7 DNS Leak Test Design

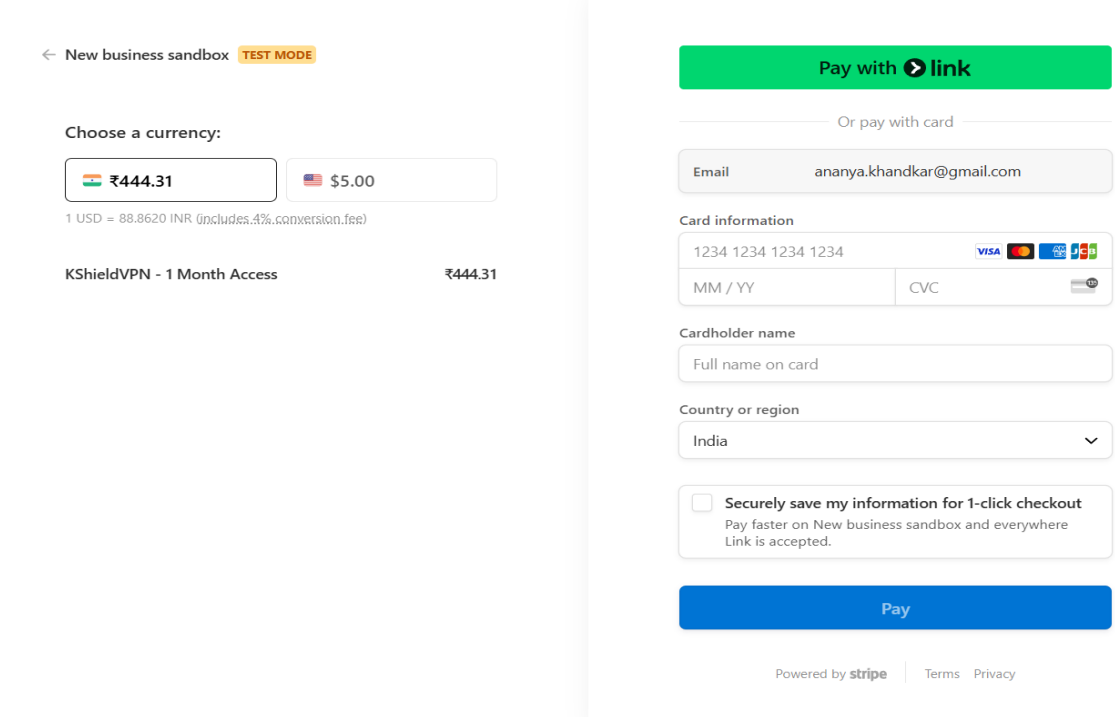


Figure 4.4.8 Payment Gateway Design

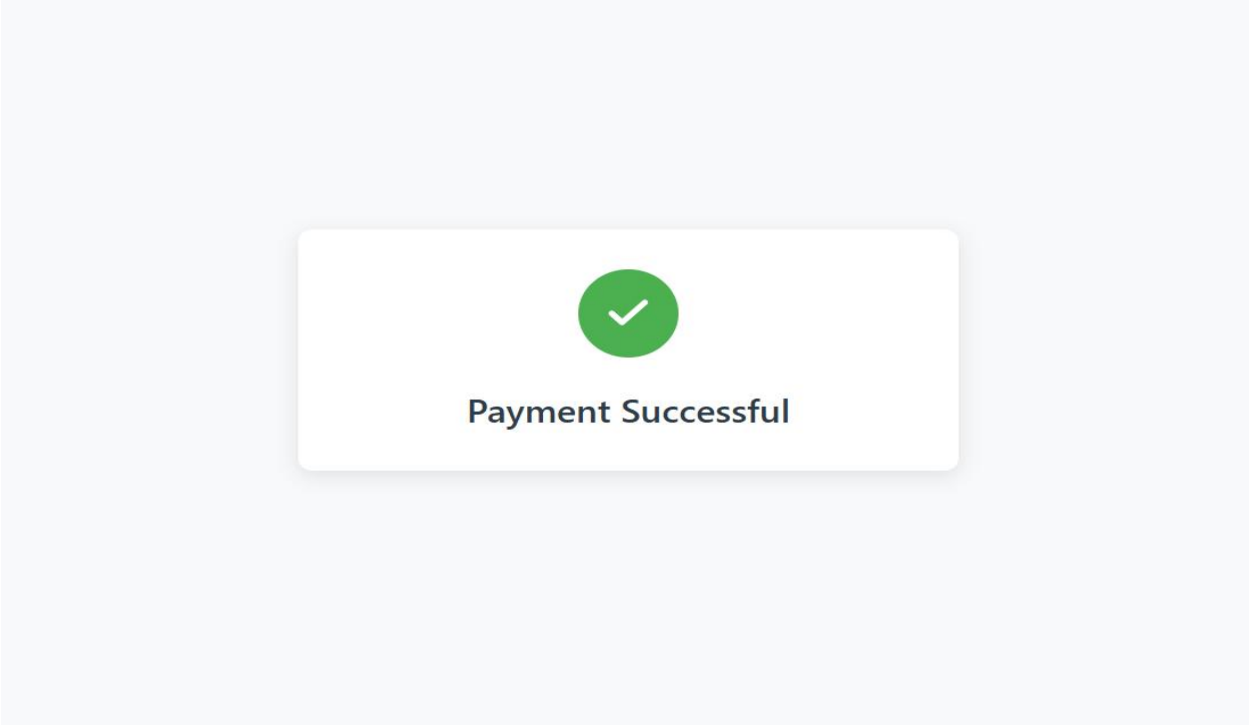


Figure 4.4.9 Payment Successful Page Design

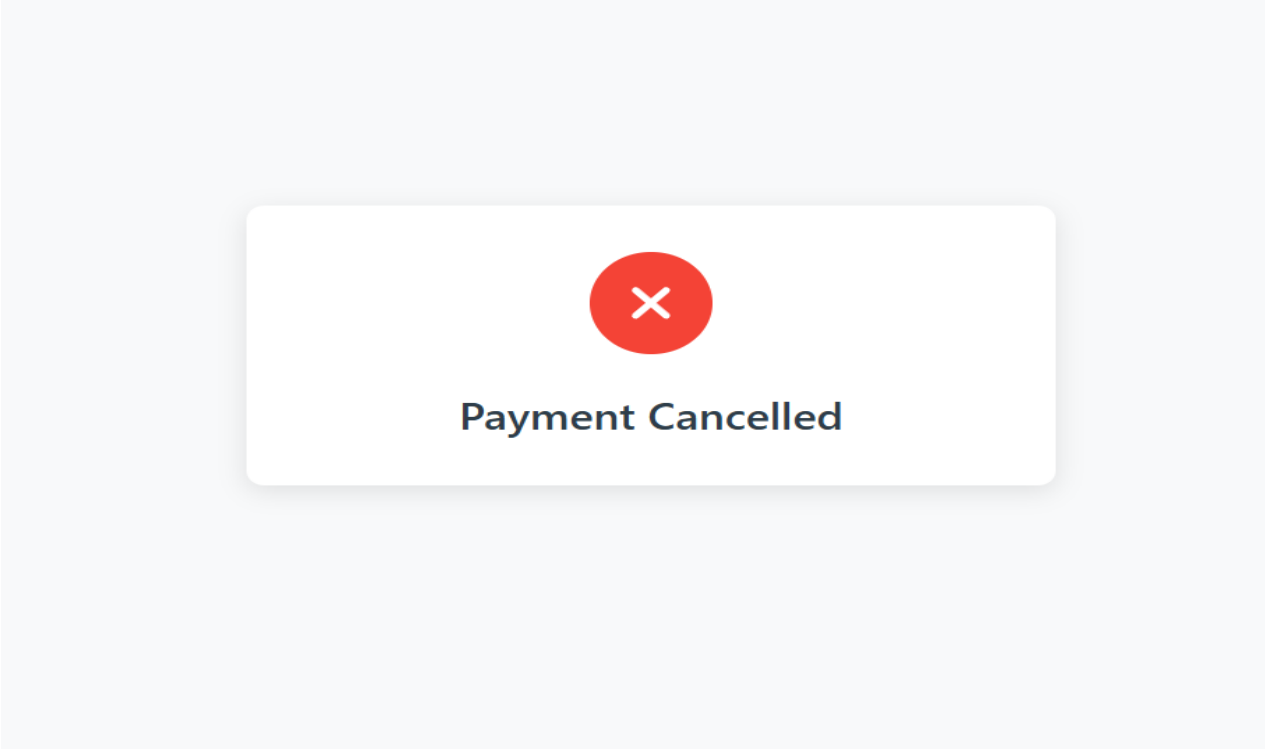


Figure 4.4.10 Payment Cancelled Page Design

4.5 Security Issues

The Security Issues aspect of the KShieldVPN system design outlines the key risks associated with VPN applications and the measures taken to mitigate them. As the system handles sensitive user credentials, network tunneling, and third-party integrations (OpenVPN), ensuring confidentiality, integrity, and controlled access is critical. Below are the major security concerns identified in KShieldVPN and how the application addresses them.

1. Plaintext Credential Storage

Storing user credentials in plaintext poses a major threat, as attackers could gain direct access to sensitive data. In KShieldVPN, all passwords are hashed using the **SHA-256** algorithm before being saved to the database. This ensures that even if the database is compromised, raw passwords cannot be easily retrieved. Additionally, the login system uses secure comparison techniques to prevent timing attacks. Users are also encouraged to use strong, complex passwords during registration.

2. OpenVPN Credential Exposure

KShieldVPN uses OpenVPN to establish VPN connections by referencing .ovpn configuration files and a separate cred.txt file. If this credentials file is not secured, it could be read or modified by malware or unauthorized users. To reduce this risk, access to the cred.txt file is limited to the application, and the file can be protected at the OS-level using file permissions. The system also avoids displaying or logging credentials anywhere in the UI or logs. Users are advised not to manually open or alter the config and credential files.

3. Lack of Encrypted Configuration Management

The .ovpn files contain sensitive information such as server IP addresses and connection methods. If tampered with, they could redirect users to rogue servers or disable encryption. KShieldVPN mitigates this by locking file editing access to admin-level users and ensuring all configuration files are pre-verified and stored in a secure location. The application does not allow uploading or importing new config files through the UI. In future enhancements, encryption of config files and integrity checks (like SHA-256 hashes) can be added.

4. Absence of Kill Switch

A kill switch ensures that if the VPN connection unexpectedly drops, the user's internet traffic is blocked to prevent unencrypted leaks. KShieldVPN does not currently implement an automated kill switch, which is a recognized security limitation. Without this, users may unknowingly transmit data over insecure channels. To reduce the risk, users are notified with

clear alerts when the VPN disconnects. The next iteration of the system may integrate a simple firewall-based kill switch using PowerShell scripts or Windows Defender Firewall rules.

5. Insecure Session Management

Sessions are crucial to controlling access, especially when differentiating between free and paid users. If session management is weak, attackers can bypass restrictions or impersonate users. KShieldVPN implements session flags upon login and clears session data on logout, including the “Remember Me” preference. Session data is stored locally using Windows Forms settings and is not exposed externally. Future upgrades may include token-based sessions and timeout-based automatic logout for enhanced security.

6. No DNS Leak Protection

A DNS leak can occur if the operating system continues to use the default ISP DNS server even while connected to a VPN. This would expose the user's browsing activity despite being connected to the VPN. KShieldVPN currently includes a **DNS Leak Test** feature via a separate form to raise user awareness. However, it does not implement active DNS leak prevention, which is a known limitation. Users are advised to configure their systems to use secure DNS manually until automatic protection is added.

7. Limited Input Validation and Logging

Input fields such as server selection and login can be susceptible to injection attacks or malformed inputs. Although the system uses dropdowns and controlled UI elements, no backend sanitization exists for server names or file paths. KShieldVPN logs OpenVPN output using Debug.WriteLine, which may capture unintended data if not filtered. This output is never shown to users but may still be readable by developers in debug mode. Implementing stricter input validation and log sanitization will reduce risks from tampered inputs or log-based attacks.

4.6 Test Case Design

The Test Cases Design aspect of the system design outlines the various scenarios the system is tested against to ensure its functionality, reliability, and robustness. For the KShieldVPN application, test cases are centered around validating the core functionalities such as user authentication, VPN connectivity, server selection, session management, and additional utility features like speed tests and DNS leak detection. Testing ensures both standard operation and edge-case handling across the application.

1. User Registration and Login

i. Test cases are designed to verify the accuracy and security of user registration and login processes.
ii. These include entering valid and invalid credentials, testing for empty or malformed inputs, duplicate user registrations, and checking SHA-256 password hashing functionality.
iii. The system should produce appropriate error messages for incorrect login attempts.
iv. Successful login should redirect the user to the VPN dashboard, while failed attempts should remain on the login page.

v. Logout functionality and re-login flow are also tested to ensure proper session management.

2. Server Selection and VPN Connection

i. Test cases validate the ability of the user to select and connect to available VPN servers.
ii. These include testing dropdown selections, verifying that only valid and configured .ovpn files are used, and ensuring proper handling of unavailable or corrupt files.
iii. Connection status should be clearly reflected in the UI, with appropriate success or failure messages.
iv. Tests include launching OpenVPN with correct parameters and capturing the connection logs.
v. Disconnection behavior is also tested to ensure the session terminates cleanly.

3. Session Management and Access Control

i. Test cases validate whether session states are correctly maintained throughout the application's lifecycle.

ii. Scenarios include accessing premium features without login, attempting to return to the dashboard after logout, and re-accessing the application without credentials.

v. Session flags and user types (free vs paid) are verified for proper feature access.

4. Speed Test Functionality

i. Tests are designed to validate the speed test form's ability to accurately retrieve and display connection speed data.

ii. The system is tested for responsiveness to multiple test runs in short intervals.
iii. Speed readings (ping, download, upload) are checked for validity, range, and format.
iv. Scenarios involving no internet connection or limited connectivity are also tested.
v. Output display is tested for clarity and real-time update capability.

5. DNS Leak Detection

i. The DNS leak test form is tested to ensure it correctly loads the external web-based leak checker.
ii. Tests include scenarios where VPN is connected and disconnected to check whether DNS behaviour changes.

iii. Error handling is tested when the DNS testing service is unavailable or slow.
iv. Clear messaging should inform users of the results or guide them to retry if needed.

6. Payment and Access Control (Demo)

i. Test cases are created for the demo payment gateway integration (e.g., Stripe).
ii. These include valid payments, cancelled transactions, and network failures during checkout.
iii. Post-payment, the system should redirect users to a success page and unlock premium features.
iv. Unauthorized users trying to access premium features are redirected appropriately.
v. Simulated test payments are also validated in developer mode to ensure sandbox isolation.

7. Error Handling and Logging

i. The application’s response to unexpected or invalid input across all modules is tested.
ii. This includes missing .ovpn files, invalid login attempts, incorrect server selections, and unsupported system configurations.

iii. Logs are checked for proper debugging information without exposing sensitive data.
iv. The system’s ability to recover from crashes or errors without data loss is also evaluated.
v. Test results are analyzed to verify that user-facing messages are informative and actionable.

Chapter 5

Implementation and Testing

This chapter presents the work flow of the project, what are the implementation approaches used, what are the coding details and efficiency, what are the testing method we can use in the project and some test case, along with modifications and improvements.

5.1 Implementation Approach:

The Implementation Approaches section outlines the strategies and methodologies used to realize the design of the KShieldVPN system. The implementation focuses on developing core functionalities such as user authentication, secure server connection, session handling, UI responsiveness, and integration with third-party tools like OpenVPN and Stripe. Each component has been built using a structured and modular approach for clarity and scalability.

1. Modular Approach

- The application is divided into several logical modules: User Authentication, Server Management, VPN Connectivity, Session Management, Speed Testing, DNS Leak Testing, and Payment Integration.
- Each module is developed independently to promote separation of concerns and easier debugging.
- This structure allows multiple parts of the project to be built and tested in parallel.
- Modules communicate via shared interfaces, ensuring maintainability and flexibility.
- Enhancements or feature updates can be isolated within their respective modules without impacting the entire system.

2. Iterative Development

- KShieldVPN was developed using an iterative approach, where each cycle includes planning, development, and testing.
- Early prototypes focused on core functionality like login and basic VPN connectivity, followed by enhancements in later iterations (e.g., speed test, DNS leak).
- This allowed for user feedback to be incorporated throughout the process, improving overall quality.

- Bugs and usability issues were identified and resolved early due to incremental development.
- The iterative model ensured continuous progress, version control, and flexibility to adapt design decisions.

3. Front-End Development

- The Windows Forms framework, along with the Guna2 UI library, was used to build a modern and responsive desktop interface.
- Focus was placed on clean navigation, intuitive buttons, and clear status indicators for connection, server status, and test results.
- Color schemes and layout were selected to improve visual clarity and user comfort.
- The design adheres to user-centered principles, offering visual feedback on all actions (e.g., successful login, VPN status).
- Navigation between forms (dashboard, login, speed test, DNS leak, etc.) is handled seamlessly using event-driven programming.

4. Back-End Development

- Backend logic is implemented in C#, managing tasks like file handling, system command execution, session tracking, and authentication.
- The application uses SQL Server to store user credentials and access information securely.
- OpenVPN connection is handled by launching .ovpn files via system commands from the backend.
- Proper exception handling and logging mechanisms are used to capture runtime issues.
- Data validation and encryption (SHA-256 for passwords) are incorporated to enhance backend reliability and security.

5. Database Implementation

- SQL Server is used to manage the system's relational database.
- Tables are created for storing user details, session logs, payment status, and access privileges.

- Referential integrity is maintained to ensure consistent and secure data management.
- Parameterized queries are used to protect against SQL injection attacks.
- The database supports login validation, session checks, and access control for premium features.

6. Integration

- The front-end Windows Forms app is integrated with backend C# logic and SQL Server for real-time data access and updates.
- OpenVPN integration is handled by invoking .ovpn configuration files through the backend using system commands.
- The Stripe payment gateway is integrated using an ASP.NET web page, linked from the app's premium section.
- Speed and DNS tests are integrated as utility features accessible from the dashboard.
- The integration ensures smooth data flow between UI actions and backend logic with minimal latency.

7. Testing

- The application is tested through unit testing (individual functions), integration testing (between modules), and system testing (end-to-end functionality).
- Login scenarios, VPN connectivity status, and error messages are verified for correctness and reliability.
- Special attention is given to validating premium access features and ensuring session integrity.
- Speed test and DNS test forms are tested under various network conditions.
- All tests are repeated under different user roles and environments to ensure robustness.

8. Deployment

- The KShieldVPN application is deployed on Windows machines with all necessary configuration files included.
- The SQL Server database is deployed locally or on a secured server based on usage needs.

- OpenVPN must be pre-installed on the host system for successful VPN connections.
- The ASP.NET web-based payment portal is hosted on a local server for demonstration purposes.
- Future deployment plans can include packaging the app with an installer and cloud-based database support.

5.2 Coding Details and Coding Efficiency

The Coding Details and Coding Efficiency section outlines the practices, standards, and strategies followed in developing the KShieldVPN system. The focus is on ensuring that the code is readable, maintainable, secure, and performs efficiently across all functionalities including user management, VPN connectivity, session handling, and payment integration.

1. Coding Standards

- The code adheres to standard C# programming conventions, including proper indentation, spacing, and consistent brace usage.
- Variables, methods, and class names follow camelCase or PascalCase conventions and are named meaningfully to reflect their purpose.
- Inline and block comments are used to describe logical steps, complex sections, and function behaviours.

2. Code Organization

- The project is structured into functional modules such as LoginForm, Kshieldvpn1, stform, and dnsleakform, each handling a specific responsibility.
- The UI layer is clearly separated from the business logic and database operations to maintain a clean separation of concerns.
- Helper methods and classes are used to encapsulate repetitive logic (e.g., OpenVPN connection, session verification, IP checks).
- Event-driven design in Windows Forms ensures that UI actions trigger well-defined functions.
  This modular organization simplifies testing, debugging, and future code modifications.

3. Efficient Algorithms

- Lightweight and optimized logic is used for repetitive operations like server switching, login verification, and VPN state checking.
- OpenVPN config file selection is dynamically mapped based on server dropdown selection to minimize redundant code.
- Session and credential checks are implemented using conditionals that execute in constant time where applicable.

- Delays or timeouts (such as waiting for VPN status) are handled with efficient polling rather than excessive looping.
- These decisions help keep the application responsive even during background operations.

4. Database Queries

- All database interactions are implemented using parameterized SQL queries to prevent SQL injection and enhance performance.
- Queries are designed to retrieve only necessary fields, avoiding heavy joins or nested subqueries that could slow down execution.
- Login, session, and payment status queries are indexed for fast access. CRUD operations are encapsulated in dedicated functions for reusability and reduced code duplication.
- The database schema is normalized to reduce redundancy and ensure consistency across all tables.

5. Error Handling

- Exception handling is implemented using try-catch blocks in all major modules including login, VPN connection, and payment handling.
- Error messages are user-friendly and provide suggestions when possible (e.g., “Check your credentials” instead of generic failures).
- Application crashes due to file issues, database errors, or VPN misconfigurations are gracefully handled.
- Logs are maintained for debugging purposes during development.
- This ensures a smoother user experience and makes maintenance easier.

6. Security

- Passwords are hashed using the SHA-256 algorithm before storing them in the database.
- User input is validated at both frontend and backend levels to protect against injection and malformed data.
- VPN access is protected by session verification to ensure only authenticated users can connect.

- Access to premium servers is restricted through session flags set only after payment confirmation.
- The application avoids storing sensitive credentials in plain text and adheres to secure coding practices.

7. Testing

- The codebase is written in a modular fashion, enabling unit testing of core logic like login, database access, and session checks.
- Each form and functionality is manually tested using a combination of black-box and white-box testing methods.
- The Arrange-Act-Assert pattern is followed while structuring test cases to ensure clarity and repeatability.
- Edge cases (e.g., blank input, invalid file path, server disconnection) are considered during testing.
- Testing ensures that the application remains reliable under various scenarios and inputs.

5.3 Testing Approach

The Testing Approach section provides a comprehensive overview of the strategies and methodologies employed to rigorously test the KShieldVPN system. The primary objective of this testing process is to ensure that the application functions as intended, delivering a seamless, secure, and reliable user experience. To achieve this, the testing was conducted at three distinct levels: unit testing , integration testing , and system testing . Each level of testing serves a specific purpose, from identifying bugs early in development to validating the complete system from the end-user's perspective. Below is an in-depth breakdown of the testing approach, including detailed test cases for each level.

Unit Testing

Unit testing focuses on verifying the functionality of individual components or modules within the KShieldVPN application. This level of testing ensures that each module performs its intended task correctly before being integrated with other components. The following test cases were designed to validate the core functionalities of the application:

1. Test Case 1: Verify User Registration Function
    - Description: Ensure that the registration logic correctly adds a new user to the database.
    - Preconditions: The database must be connected, and no duplicate username or email should exist.
    - Steps:
        1. Navigate to the registration form.
        2. Fill in valid details, including a unique username, email, and password.
        3. Click the “Register” button.
        4. Check the database for the newly added record.
    - Expected Result: The user is successfully added to the database, and a success message is displayed to the user. The stored password should be hashed using SHA-256 and not stored in plaintext.

2. Test Case 2: Validate User Login with Correct Credentials
    - Description: Ensure that users can log in successfully when providing valid credentials.
    - Preconditions: The user must already be registered in the database.
    - Steps:
        1. Enter valid login credentials (email and password).
        2. Click the “Login” button.
    - Expected Result: The user is authenticated, logged in, and redirected to the dashboard. The session state should be active, allowing the user to access the application’s features.
3. Test Case 3: Handle Login with Incorrect Credentials
    - Description: Ensure that incorrect login attempts are handled gracefully, with appropriate error messages displayed to the user.
    - Preconditions: The user attempts to log in with invalid credentials (e.g., wrong password or non-existent email).
    - Steps:
        1. Enter invalid login credentials.
        2. Click the “Login” button.
    - Expected Result: The login attempt fails, and the user is shown an error message indicating that the credentials are incorrect. The system should prevent unauthorized access and maintain security.
4. Test Case 4: Verify VPN Configuration Loading
    - Description: Ensure that the correct OpenVPN configuration file (.ovpn) is loaded based on the selected server.
    - Preconditions: The OpenVPN configuration files must exist in the designated config folder.
    - Steps:
        1. Select a server location (e.g., Seoul or Paris) from the dropdown menu.

2. Click the “Connect” button.

- Expected Result: The corresponding .ovpn file for the selected server is loaded into the OpenVPN process. The system should display a confirmation message indicating that the connection process has started.

5. Test Case 5: Test Password Hashing Mechanism

- Description: Ensure that the password hashing mechanism produces consistent and secure output using the SHA-256 algorithm.
- Preconditions: None.
- Steps:

    1. Register a new user by filling out the registration form.
    2. Manually verify the stored password hash in the database.

- Expected Result: The password stored in the database should be hashed using SHA-256 and should not be stored in plaintext. The hashing process should be irreversible, ensuring the security of user credentials.

Integration Testing

Integration testing focuses on verifying that individual modules work together seamlessly. This level of testing ensures that the interactions between components, such as the database, user interface, and external services like OpenVPN, function as expected. The following test cases were designed to validate the integration of key features:

6. Test Case 6: Test Dashboard Navigation Post-Login

- Description: Ensure that successful login leads to the dashboard and that the session state is properly initialized.
- Preconditions: Valid user credentials must exist in the database.
- Steps:

    1. Log in using valid credentials.
    2. Observe the post-login behavior.

- Expected Result: The user is redirected to the KShieldVPN dashboard, and the session state is active. The dashboard should display all available features, including server selection, speed test, and DNS leak test options.

7. Test Case 7: Test Server Selection and VPN Start Flow

    - Description: Ensure that selecting a VPN server initiates a valid OpenVPN connection.
    - Preconditions: The user must be logged in, and the OpenVPN configuration files must be available in the designated folder.
    - Steps:
        1. Choose a server location from the dropdown menu.
        2. Click the “Connect” button.
        3. Monitor the connection log for activity.
    - Expected Result: The VPN connection is successfully established, and the status indicator shows “Connected.” The system should provide feedback to the user confirming the successful connection.

8. Test Case 8: Test Logout and Session End

    - Description: Ensure that the user session ends and access is revoked upon logging out.
    - Preconditions: The user must be logged in.
    - Steps:
        1. Click the “Logout” button.
        2. Attempt to navigate back to the dashboard or access restricted features.
    - Expected Result: The user is redirected to the login page, and the dashboard becomes inaccessible. The session state should be terminated, ensuring that unauthorized access is prevented.

System Testing

System testing evaluates the complete functionality of the KShieldVPN application from an end-to-end perspective. This level of testing ensures that the application meets the requirements and provides a smooth user experience across all features. The following test cases were designed to validate the overall system performance:

9. Test Case 9: Simulate Payment and Access to Premium Servers
    - Description: Verify that only paid users can access premium VPN servers and features.
    - Preconditions: A payment session must be simulated as complete (e.g., using a demo payment gateway).
    - Steps:
        1. Complete the payment process via the integrated payment gateway.
        2. Attempt to access premium server locations and advanced features.
    - Expected Result: Access to premium servers and features is granted only to users whose payment session has been verified. Free users should remain restricted to basic access.
10. Test Case 10: End-to-End VPN Connection and Disconnection
    - Description: Test the full cycle of connecting to and disconnecting from a VPN server through the user interface.
    - Preconditions: The user must be logged in, and the OpenVPN setup must be complete.
    - Steps:
        1. Select a server location from the dropdown menu and click “Connect.”
        2. Browse the internet for a few minutes to verify the connection.
        3. Click the “Disconnect” button.
    - Expected Result: The VPN connects successfully, and the user’s internet traffic is routed through the selected server. Upon disconnecting, the system gracefully terminates the connection, returning the user to their original network state. The status indicator should update accordingly.

Additional Considerations

- Error Handling and Notifications:

  Throughout the testing process, special attention was given to error handling mechanisms. For example, if a user encounters an issue during the VPN connection process, the system provides clear and actionable feedback. Similarly, error messages during login attempts or server selection are designed to guide users in resolving issues effectively.

- Performance Testing:

  In addition to functional testing, performance testing was conducted to evaluate the application's responsiveness under various conditions. This included measuring the time taken to establish a VPN connection, load test results, and handle multiple simultaneous operations.

- Security Testing:

  Security testing was a critical component of the overall testing strategy. The password hashing mechanism, session management, and data encryption were thoroughly validated to ensure that sensitive user information remains protected at all times.

5.4 Modifications and Improvements

The **Modifications and Improvements** section outlines changes made during implementation based on testing results, performance issues, and user feedback. It also highlights potential enhancements that can be incorporated in future iterations to improve the functionality, security, usability, and performance of the **KShieldVPN** system.

1. User Management

- **Modification**: Improved session handling to prevent unauthorized access to premium features after logout.
- **Future Improvement**: Implement role-based access control (e.g., admin vs. user) to better manage different levels of user privileges.
- **Future Improvement**: Add multi-factor authentication (MFA) for enhanced login security.

2. VPN Server Management

- **Modification**: Refined the server selection logic to dynamically load the appropriate OpenVPN config files based on user selection.
- **Future Improvement**: Introduce a "Smart Connect" feature that automatically selects the fastest available server based on ping or server load.
- **Future Improvement**: Add an option to favourite servers for quicker access.

3. Session & Access Control

- **Modification**: Strengthened session verification across all forms, ensuring users cannot access premium features without a valid session.
- **Future Improvement**: Include session timeout and auto-logout after inactivity to prevent misuse.
- **Future Improvement**: Maintain a session history log showing login and VPN usage activity.

4. Connectivity Features

- **Modification**: Streamlined OpenVPN command execution and added error prompts for missing or invalid configuration files.
- **Future Improvement**: Integrate a kill switch mechanism to block internet access when VPN is disconnected unexpectedly.
- **Future Improvement**: Add built-in DNS leak protection and IPv6 blocking to strengthen privacy.

5. UI and User Experience

- **Modification**: Refined navigation flow and UI responsiveness across different forms.
- **Future Improvement**: Redesign UI with a more modern, responsive layout using dark/light themes.
- **Future Improvement**: Add tooltips and onboarding popups to help new users understand each feature.

6. Payment & Subscription System

- **Modification**: Integrated payment flow using Stripe to restrict premium features to paid users.
- **Future Improvement**: Include subscription management dashboard where users can view billing history and update payment methods.
- **Future Improvement**: Add support for trial plans or usage-based pricing for flexibility.

7. Performance & Scalability

- **Modification**: Optimized database interactions to speed up login and server selection processes.
- **Future Improvement**: Implement connection logs with real-time updates, useful for admins and debugging.
- **Future Improvement**: Use caching mechanisms and async operations to improve responsiveness under high usage.

5.5 Test Cases

5.5.1 General test cases

Test Case ID	Description	Field	Expected Pattern	Invalid Input
TC_001	Invalid email format entered	Email	Valid email format (e.g., example@example.com)	example@, example.com
TC_002	Password field left empty	Password	At least 8 characters long	*(Empty)*
TC_003	Password less than 8 characters entered	Password	At least 8 characters long	pass, 1234567
TC_004	Invalid server selected	Server Selection	Predefined server options	Custom, Unknown
TC_005	Empty username during login	Username	Valid registered username	*(Empty)*
TC_006	Empty password during login	Password	Valid password	*(Empty)*
TC_007	Incorrect credentials during login	Username/Password	Matching registered credentials	WrongUsername / WrongPassword

TC_008	Session expires after logout	Session Token	Active session token	Expired or removed session token
TC_009	Invalid payment details during Stripe checkout	Card Info	Valid card number, expiry, and CVV	1234 5678 0000 0000, expired date
TC_010	Accessing premium features without login	Session	Valid active session	No session token / Not logged in

5.5.2 Unit Testing

Test Case ID	Description	Preconditions	Steps	Expected Result
TC_U 001	Verify login with correct credentials	User already registered with hashed password in DB	1. Open login form 2. Enter valid email and password 3. Click Login	User is redirected to main form; session and premium flags are set
TC_U 002	Verify login with incorrect password	User exists in DB	1. O pen login form 2. Enter valid email and incorrect password 3. Click Login	Error message shown: "Invalid password"
TC_U 003	Login with non-existent email	Email not present in DB	1. Open login form 2. Enter non-registered email and any password 3. Click Login	Error message shown: "No account available with this email"
TC_U 004	Login with empty fields	-	1. Open login form 2. Leave email or password empty	Error message: "Please enter value in all fields."

			3. Click Login	
TC_U 005	Register new user with matching passwords	Email not already in DB	1. Open register form	User is added to DB with hashed password; redirected to login
			2. Enter email	
			3. Enter password & confirm password	
			4. Click Register	
TC_U 006	Register with mismatched passwords	-	1. Open register form	Error message: "Password doesn't match"
			2. Enter passwords that don’t match	
			3. Click Register	
TC_U 007	Register with empty fields	-	1. Open register form	Error message: "Please enter value in all field."
			2. Leave any field empty	
			3. Click Register	
TC_U 008	HashPassword generates consistent hash	-	Call Form2.HashPassword("test123") twice	Both outputs are identical

TC_U 009	VerifyPassword returns true for matching hash	Password already hashed	Call VerifyPassword("test123", HashPassword("test123"))	Returns true
TC_U 010	VerifyPassword returns false for non-matching hash	-	Call VerifyPassword("wrong", HashPassword("test123"))	Returns false
TC_U 011	Successful VPN connection to Seoul	OpenVPN installed and config file exists	1. Launch main form 2. Select "Seoul" 3. Click Connect	VPN process is launched; success message is shown
TC_U 012	VPN connection attempt without server selection	-	1. Launch main form 2. Leave server dropdown empty 3. Click Connect	Error message: "Select server"
TC_U 013	Disconnect VPN	VPN already running	Click Disconnect	openvpn.exe process is terminated
TC_U 014	Menu toggle expands and collapses UI panel	Main form loaded	Click hamburger menu icon	Panel expands or fades away

TC_U 015	Logout clears session and opens login form	User logged in	Click Logout	Session cleared; login form opened
TC_U 016	CheckIfUserIsPre mium returns true	User is premium in DB	Call CheckIfUserIsPremium("pre mium@example.com")	Returns true
TC_U 017	CheckIfUserIsPre mium returns false	User not premium in DB	Call CheckIfUserIsPremium("basi c@example.com")	Returns false
TC_U 018	Payment button opens Stripe Checkout	Email exists in settings	Click Premium button	Browser opens Stripe session URL
TC_U 019	Load form with remembered login	UserEmail saved in settings	Launch login form	Redirects to main form without login
TC_U 020	Menu animation fades out correctly	Menu already expanded	Click hamburger icon	Menu fades out and hides

5.5.3 Integration testing

Test Case ID	Description	Preconditions	Steps	Expected Result
TC_I001	Login Form → Main Form transition with valid credentials	User registered in DB	1. Launch app 2. Enter valid credentials 3. Click Login	Main form opens; session flags are passed correctly
TC_I002	Register Form → Login Form redirection after successful registration	Email not already registered	1. Open register form 2. Register with valid inputs 3. Click Register	New user created; login form opens
TC_I003	Main Form → OpenVPN → External Config Integration	Config file exists and valid	1. Login successfully 2. Select server 3. Click Connect	openvpn.exe launched with correct config and auth file
TC_I004	VPN Disconnect → Process termination	VPN is connected	1. VPN running via openvpn.exe 2. Click Disconnect	VPN process is terminated cleanly

TC_I005	Main Form → Stripe Payment Integration	User is logged in	1. Click Premium button 2. Stripe window opens	Stripe checkout session is created; user redirected to Stripe
TC_I006	Premium purchase → User promoted to premium in DB → UI updated	Payment success and webhook updates DB	1. Complete Stripe payment 2. Open app again 3. Login	UI detects premium status; premium features accessible
TC_I007	Menu interaction → Dynamic panel behavior	User is on Main Form	Click hamburger menu icon	Panel expands/collapses using animation
TC_I008	Login → Session flags passed to dashboard	User credentials valid	1. Login 2. Dashboard opens	SessionManager.UserEmail and .IsPremiumUser set correctly
TC_I009	Premium status check → Access control for features	User is basic (not premium)	Try accessing premium-only features	Access denied or prompt shown to upgrade
TC_I010	SpeedTest Form integration	User on main form	1. Click "Speed Test"	stform opens with correct data handling

			2. New form opens	
TC_I011	DNS Leak Form integration	User on main form	1. Click "DNS Leak Test" 2. Form opens	dnsleakform opens and loads relevant UI
TC_I012	Logout → Clears session and returns to login	User is logged in	Click Logout	Session flags reset; login form shown
TC_I013	Login failure → UI not redirected	Credentials invalid	Enter wrong password and click Login	Error shown; no redirection
TC_I014	Register → DB Insert + Form Redirect	Unique email entered	Complete registration	DB updated; redirected to login form

5.5.4 System Testing

Test Case ID	Description	Preconditions	Steps	Expected Result
TC_S001	Complete user journey: Register → Login → Connect VPN	App running	1. Register 2. Login 3. Select server 4. Click Connect	VPN connection established end-to-end
TC_S002	Invalid login attempt from login screen	App running	1. Enter wrong credentials 2. Click Login	Error message displayed, no access
TC_S003	Attempt to connect VPN without selecting server	User logged in	1. Click Connect without server selection	Error shown or no action taken
TC_S004	Connect to VPN → Disconnect → Reconnect	Valid credentials	1. Login 2. Connect 3. Disconnect 4. Connect again	VPN connects and disconnects properly every time
TC_S005	Payment and premium upgrade end-to-end	Valid user, payment gateway live	1. Login 2. Open Premium	User upgraded to premium, new features enabled

			3. Complete Stripe payment	
TC_S006	Premium features restriction for basic users	Logged in as basic user	Try using a premium feature (e.g., select premium-only server)	Access blocked with upgrade prompt
TC_S007	Navigation across all forms	App running	Click buttons to open DNS test, speed test, logout, etc.	All forms load and respond properly
TC_S0010	Session timeout or manual logout → Access blocked	User logged out or session cleared	1. Logout 2. Try using back navigation or reopening main form	User redirected to login; session not restored
TC_S011	All buttons and animations respond correctly	Logged in	Click all buttons: menu toggle, dashboard, DNS test, etc.	All elements animate/work correctly
TC_S012	Email duplication check during registration	Email already exists	Try registering with same email	Error message shown, no duplicate allowed

Chapter 6

Results and Discussion

This chapter presents test reports and user documentation based on the above chapter's test cases.

6.1 Test Reports

6.1.1 General Test Cases

TC_001 – Invalid Email Format

Discussion: When an improperly formatted email is entered during registration, the system should validate and reject it.

Result: Error message "Invalid email format" displayed.

TC_002 – Password Field Left Empty

Discussion: Leaving the password field blank during registration or login should trigger a required field validation.

Result: Error message "Password is required" displayed.

TC_003 – Password Less Than 8 Characters

Discussion: Short passwords that don't meet minimum security requirements must be rejected.
Result: Error message "Password must be at least 8 characters long".

TC_004 – Invalid Server Selected

Discussion: When a user tries selecting a non-existent or unknown VPN server, the app should not proceed.

Result: Error message "Invalid server selection" displayed.

TC_005 – Empty Username During Login

Discussion: If the username is left empty, login should be blocked and an alert shown.
Result: Error message "Username is required" displayed.

TC_006 – Empty Password During Login

Discussion: The login process must enforce entry of the password field. Result: Error message "Password is required" displayed.

TC_007 – Incorrect Credentials During Login

Discussion: When wrong login details are provided, the app must not authenticate the user. Result: Error message "Invalid credentials" displayed.

TC_008 – Session Expires After Logout

Discussion: After logout, access to protected content must be restricted until a new login. Result: Redirected to login page successfully.

TC_009 – Invalid Payment Details During Stripe Checkout

Discussion: Fake or expired payment card info should not be accepted at checkout. Result: Error message "Invalid payment details" displayed.

TC_010 – Accessing Premium Features Without Login

Discussion: Users not logged in should be restricted from accessing premium content. Result: Access denied or redirected to login page.

6.1.2 Unit Test Cases

TC_U001 – Verify Login with Correct Credentials

Discussion: When a registered user enters valid email and password, the system should authenticate successfully, initialize the session, and provide access to the main application interface.
Result: The user is redirected to the main form, and both session and premium status flags are set correctly.

TC_U002 – Verify Login with Incorrect Password

Discussion: If a user inputs a correct email but an incorrect password, the system should block access and provide a relevant error message.

Result: An error message "Invalid password" is displayed, and access to the system is denied.

TC_U003 – Login with Non-existent Email

Discussion: Attempting to log in with an email address that is not present in the database should result in an error message without further processing.

Result: An error message "No account available with this email" is shown to the user.

TC_U004 – Login with Empty Fields

Discussion: If either the email or password fields are left blank during login, the system should prompt the user to fill in all required fields.

Result: A validation error "Please enter value in all fields." is displayed.

TC_U005 – Register New User with Matching Passwords

Discussion: When a user provides a new email and matching passwords, the system should create an account with a securely hashed password and redirect to the login page.
Result: The user is successfully registered, stored in the database with a hashed password, and redirected to the login screen.

TC_U006 – Register with Mismatched Passwords

Discussion: If the password and confirm password fields do not match, registration should be halted with a clear alert message.

Result: The user is shown an error message: "Password doesn't match", and registration is blocked.

TC_U007 – Register with Empty Fields

Discussion: Registration must not proceed if any required fields (email, password, or confirmation) are left empty.

Result: The system shows an error message: "Please enter value in all field." and does not submit the form.

TC_U008 – HashPassword Generates Consistent Hash

Discussion: Calling the hashing function with the same input multiple times should consistently produce the same hash value.

Result: Both calls to the function return identical hash strings, confirming consistency.

TC_U009 – VerifyPassword Returns True for Matching Hash
Discussion: The password verification method should correctly validate a plaintext password against its corresponding hash.

Result: The method returns true, indicating a successful password match.

TC_U010 – VerifyPassword Returns False for Non-matching Hash
Discussion: A password input that does not match the original hashed value must be rejected during verification.

Result: The method returns false, correctly identifying a mismatch.

TC_U011 – Successful VPN Connection to Seoul
Discussion: Selecting the Seoul server and clicking connect should launch OpenVPN and establish a successful VPN connection.

Result: The OpenVPN process is started, and a success message is displayed to the user.

TC_U012 – VPN Connection Attempt Without Server Selection
Discussion: The user should not be allowed to initiate a VPN connection without selecting a valid server from the list.

Result: An error message "Select server" is shown, preventing the VPN connection.

TC_U013 – Disconnect VPN

Discussion: Clicking the disconnect button while the VPN is active should terminate the OpenVPN process.

Result: The openvpn.exe process is successfully terminated, and the connection is closed.

TC_U014 – Menu Toggle Expands and Collapses UI Panel
Discussion: The hamburger menu should toggle the side panel's visibility with a smooth animation effect.

Result: The side panel expands or fades away based on its current state.

TC_U015 – Logout Clears Session and Opens Login Form
Discussion: Upon clicking logout, the system should terminate the session and navigate the user back to the login screen.

Result: Session information is cleared, and the user is redirected to the login form.

TC_U016 – CheckIfUserIsPremium Returns True

Discussion: The method must return true when querying a user marked as premium in the database.
Result: The method returns true, confirming the premium status of the user.

TC_U017 – CheckIfUserIsPremium Returns False

Discussion: If the user is not marked as premium in the database, the method should return false.
Result: The method returns false, indicating that the user is not a premium member.

TC_U018 – Payment Button Opens Stripe Checkout

Discussion: Clicking the premium button should trigger the browser to open a Stripe Checkout session using the configured URL.

Result: The browser launches and loads the Stripe payment page as expected.

TC_U019 – Load Form with Remembered Login

Discussion: If the user previously selected to remember their login, launching the app should skip the login screen and open the main form directly.
Result: The user is redirected to the main form without having to log in again.

TC_U020 – Menu Animation Fades Out Correctly

Discussion: When the expanded side menu is toggled off, it should fade out smoothly and become hidden from view.

Result: The menu fades out and collapses, maintaining a consistent user experience.

6.1.3 Integration testing cases

- TC_I001 – Login Form → Main Form Transition with Valid Credentials
  Discussion: After successful login, the application should transition from the login form to the main dashboard while passing user session data correctly.
  Result: The main form opens, and session flags such as user email and premium status are correctly initialized.

- TC_I002 – Register Form → Login Form Redirection After Successful Registration
  Discussion: Upon registering a new account with valid inputs, the app should redirect the user to the login screen for further authentication.
  Result: A new user is created in the database, and the system redirects the user to the login form.

- TC_I003 – Main Form → OpenVPN → External Config Integration
  Discussion: Selecting a server and connecting from the main form should launch OpenVPN with the correct configuration and authentication details.
  Result: The openvpn.exe process is launched using the correct config and credential files, establishing a VPN connection.

- TC_I004 – VPN Disconnect → Process Termination
  Discussion: When the user disconnects the VPN, the OpenVPN process should terminate gracefully without errors or lingering background processes.
  Result: The OpenVPN process is cleanly terminated, confirming proper disconnection functionality.

- TC_I005 – Main Form → Stripe Payment Integration
  Discussion: Clicking the Premium button while logged in should redirect the user to the Stripe Checkout page for payment processing.
  Result: A Stripe session is created, and the browser is launched with the Stripe payment interface.

- TC_I006 – Premium Purchase → User Promoted to Premium in DB → UI Updated
  Discussion: After a successful payment, the webhook should update the user's premium status in the database, and the app should reflect this on the next login.
  Result: The app detects the premium status from the database, and premium features become accessible upon login.

- TC_I007 – Menu Interaction → Dynamic Panel Behavior
  Discussion: Toggling the hamburger menu should dynamically expand or collapse the side panel with a smooth animation effect.

Result: The side panel responds correctly to user interaction, expanding or collapsing with a transition.

- TC_I008 – Login → Session Flags Passed to Dashboard
  Discussion: After login, session data such as user email and premium status must be passed accurately to the main form and stored in SessionManager.
  Result: The SessionManager.UserEmail and .IsPremiumUser values are correctly set and accessible in the dashboard.

- TC_I009 – Premium Status Check → Access Control for Features
  Discussion: Basic users should not have access to premium features and should be either denied or prompted to upgrade if they attempt access.
  Result: Access is blocked, and the user sees a message asking to upgrade to access premium features.

- TC_I010 – SpeedTest Form Integration

  Discussion: Clicking on the "Speed Test" button should open the stform and correctly load any relevant data required for the test.

  Result: The stform opens successfully, displaying the appropriate UI and handling inputs or outputs as expected.

- TC_I011 – DNS Leak Form Integration

  Discussion: The DNS Leak Test feature should launch the dnsleakform and display the related controls and UI components.

  Result: The dnsleakform launches correctly and is ready to perform or display DNS leak test information.

- TC_I012 – Logout → Clears Session and Returns to Login
  Discussion: Clicking the logout button should clear the session variables and safely return the user to the login screen.

  Result: Session data is reset, and the login form appears for the user to re-authenticate.

- TC_I013 – Login Failure → UI Not Redirected
  Discussion: If the user provides invalid credentials, the system should show an error and not allow redirection to the main form.

  Result: An error message is shown, and the application remains on the login screen.

- TC_I014 – Register → DB Insert + Form Redirect
  Discussion: On completing the registration form with a unique email, the system should insert the user into the database and navigate back to the login form.
  Result: The database is updated with the new user, and the app redirects to the login screen as expected.

6.1.4 System Testing

TC_S001 – Complete User Journey: Register → Login → Connect VPN
Discussion: This test covers the full user flow from account creation to establishing a secure VPN connection, ensuring all key components are integrated properly.
Result: The user successfully registers, logs in, selects a server, and connects to the VPN, confirming smooth end-to-end functionality.

TC_S002 – Invalid Login Attempt from Login Screen
Discussion: The system should detect incorrect credentials and prevent unauthorized access while providing the appropriate error feedback.
Result: An error message is displayed indicating invalid credentials, and the user is not allowed to access the main dashboard.

TC_S003 – Attempt to Connect VPN Without Selecting Server
Discussion: When the user clicks Connect without choosing a VPN server, the app should alert them or block the action to avoid an undefined VPN launch.
Result: No connection is made, and the application prompts the user to select a server before proceeding.

TC_S004 – Connect to VPN → Disconnect → Reconnect

Discussion: This test ensures that the VPN can be connected, disconnected, and reconnected seamlessly without crashing or leaving orphaned processes.
Result: The VPN connects and disconnects smoothly each time, and reconnection works without any functional or UI issues.

TC_S005 – Payment and Premium Upgrade End-to-End

Discussion: Once payment is completed through Stripe, the system should upgrade the user to premium and unlock additional features.

Result: After payment completion, the user's premium status is updated in the database, and the interface reflects access to premium content.

TC_S006 – Premium Features Restriction for Basic Users
Discussion: Users without premium access should be restricted from using advanced features

and prompted to upgrade if they attempt to access them. Result: Access is denied, and the system displays a message prompting the user to upgrade to premium.

TC_S007 – Navigation Across All Forms

Discussion: Clicking buttons to navigate to DNS Leak Test, Speed Test, or Logout should correctly open the respective forms and ensure all components load as intended. Result: All secondary forms open and function correctly, validating the application's internal navigation and responsiveness.

TC_S010 – Session Timeout or Manual Logout → Access Blocked Discussion: After a session ends via logout or timeout, accessing the main form through back navigation should not be possible.

Result: The user is redirected to the login page, and access to protected areas without a valid session is blocked.

TC_S011 – All Buttons and Animations Respond Correctly Discussion: Every interactive element like the menu toggle, navigation icons, and feature buttons should be responsive and display proper animations.

Result: All buttons work as expected, and UI elements animate correctly, maintaining smooth user interaction.

TC_S012 – Email Duplication Check During Registration Discussion: The system should prevent the creation of multiple accounts using the same email by validating entries during registration.

Result: An appropriate error message is shown when a duplicate email is entered, and the registration is blocked.

6.2 User Documentation

The **KShieldVPN** application is a secure and user-friendly VPN platform designed to provide safe internet access through various international servers. This guide walks users through the key functionalities of the application, including registration, login, VPN usage, and premium access.

1. Installing and Launching the Application

- Launch the **KShieldVPN** application by double-clicking the application icon.
- The application will open to the **Login** screen.

2. Registering a New Account

- Click on the **Register** button if you are a new user.
- Fill in your **email**, **password**, and **confirm password** fields.
- Click **Register** to create your account.
- If registration is successful, you will be redirected to the **Login** screen.
- If passwords do not match or fields are empty, an error message will be shown.

3. Logging In

- Enter your registered **email** and **password** on the **Login** screen.
- Click **Login**.
- If your credentials are correct, you will be taken to the **Main Dashboard**.
- If the credentials are incorrect or the account does not exist, an appropriate error message will be displayed.

4. Forgot Password

- If you forget your password, click the **Forgot Password** link on the login screen.
- Enter your registered email address.
- You will receive an email with instructions to reset your password.
- After resetting, you can return to the application and log in with the new password.

5. Connecting to a VPN Server

- Once logged in, the **Main Dashboard** will be displayed.
- Use the dropdown menu to select a server (e.g., **Seoul**, **Paris**).
- Click the **Connect** button.
- If the connection is successful, you will receive a confirmation message.
- Your internet traffic will now be securely routed through the selected VPN server.

6. Disconnecting the VPN

- To disconnect from the VPN, click the **Disconnect** button.
- The VPN connection will be terminated and you will return to your normal network connection.

7. Premium Features

- Some VPN servers and features are reserved for **Premium users**.
- To upgrade, click the **Go Premium** button on the dashboard.
- You will be redirected to a secure payment page powered by **Stripe**.
- After successful payment, your account will be upgraded automatically.
- Once upgraded, you will gain access to premium servers and additional tools.

8. Additional Tools

- **Speed Test**: Access this feature from the dashboard to measure your current internet speed.
- **DNS Leak Test**: Check if your DNS requests are leaking outside the VPN tunnel.
- These tools can be launched from the left-side navigation panel.

9. Logging Out

- To log out of your session, click the **Logout** button on the dashboard.
- This action will clear your session and return you to the login screen.

10. Common Error Messages

- **"Please enter value in all fields"** – One or more required fields were left empty.
- **"Invalid email format"** – The entered email is not in the correct format.
- **"Password doesn't match"** – The confirmed password does not match the original password.
- **"Invalid credentials"** – Email or password entered is incorrect.
- **"Select server"** – You attempted to connect without selecting a VPN server.

Chapter 7

Conclusions

This chapter presents what we can conclude from the project, what are the limitations of the system and also what future work can be applied in future.

7.1 Conclusion

The development of KShieldVPN represents a comprehensive and user-centric approach to providing secure internet browsing through the use of Virtual Private Network (VPN) technology. The application successfully integrates an intuitive Windows Forms interface with robust backend functionalities, including user authentication, secure VPN connectivity, and premium access management. At its core, KShieldVPN emphasizes security and privacy by implementing industry-standard practices such as password hashing using the SHA-256 algorithm and session management to prevent unauthorized access. These measures ensure that sensitive user data is handled responsibly, forming the foundation of a secure system. Additionally, features like the Forgot Password functionality enhance usability by empowering users to recover account access independently, reducing reliance on external support and improving the overall user experience.

One of the standout features of KShieldVPN is its seamless integration with OpenVPN, enabling users to connect to servers across the globe with a single click. This functionality allows users to protect their online privacy, bypass regional restrictions, and access geo-blocked content while ensuring that only valid server configurations are accepted, minimizing the risk of misconfiguration or misuse. To further enhance user confidence, the application includes diagnostic tools such as the Speed Test and DNS Leak Test modules. These tools allow users to verify the performance and privacy of their connections, identifying potential vulnerabilities like DNS leaks and ensuring that their online activities remain secure. By incorporating these advanced features, KShieldVPN goes beyond basic connectivity, offering a more comprehensive solution for privacy-conscious users.

To cater to diverse user needs, KShieldVPN introduces a tiered access model that distinguishes between free and premium users. Premium features, such as access to advanced server locations, enhanced speed tests, and additional privacy tools, are unlocked through a subscription system integrated with the Stripe payment gateway. This real-world simulation of a subscription model adds practical value to the application and demonstrates how monetization strategies can be effectively implemented without compromising usability or security. The system accurately differentiates between basic and premium users, enabling or

restricting features accordingly, ensuring that users receive the appropriate level of access based on their subscription status. This feature not only highlights the application's commercial potential but also reinforces its scalability and adaptability for future enhancements.

The modular architecture of KShieldVPN is another key strength, contributing to its scalability, maintainability, and ease of testing. The application is divided into distinct modules, each responsible for specific functionalities such as login, registration, dashboard navigation, speed testing, and DNS leak detection. This separation of concerns ensures that individual components can be developed, tested, and updated independently, reducing the risk of introducing bugs or inconsistencies during updates. Rigorous testing methodologies were employed throughout the development process, including unit testing to validate individual modules, integration testing to ensure seamless interaction between components, and system testing to evaluate the complete application from an end-user perspective. This multi-layered approach helped identify and address potential issues early, ensuring a high-quality user experience.

KShieldVPN places a strong emphasis on user experience, leveraging modern design principles and tools to create a clean, intuitive, and visually appealing interface. The application utilizes Guna2 controls to achieve a polished and professional look, with smooth transitions, animations, and feedback messages that guide users through their interactions. For example, when connecting to a VPN server, users receive real-time feedback, such as progress indicators and success messages, informing them of the connection status. Similarly, error messages are presented in a clear and actionable manner, helping users troubleshoot issues effectively. This attention to detail makes the application accessible even to users unfamiliar with VPN technology, broadening its appeal and usability.

While KShieldVPN was initially developed as a demonstration project, it has significant potential for real-world deployment in academic, personal, or commercial settings. Its modular architecture and scalable design make it adaptable to a wide range of use cases, from educational institutions seeking to teach students about cybersecurity to individuals looking for a reliable tool to protect their online privacy. With proper hosting infrastructure and additional enhancements, such as role-based access control, multi-factor authentication, and advanced logging capabilities, the application could evolve into a robust and feature-rich solution for privacy-conscious users. Features like the Speed Test and DNS Leak Test modules position KShieldVPN as more than just a basic VPN client—it is a comprehensive privacy toolkit designed to meet the needs of modern users.

Looking ahead, there are numerous opportunities to expand and enhance KShieldVPN. Potential improvements include introducing role-based access control (RBAC) to manage user accounts and monitor system activity, adding multi-factor authentication (MFA) for an

extra layer of login security, and implementing a "Smart Connect" feature that automatically selects the fastest available server based on ping time or server load. Other enhancements could include a kill switch mechanism to block internet access when the VPN connection is unexpectedly terminated, customizable dark/light themes for UI personalization, and a subscription management dashboard where users can view billing history and update payment methods independently. These enhancements would further solidify KShieldVPN's position as a versatile and user-friendly solution for secure internet browsing.

In conclusion, KShieldVPN is a practical and well-rounded demonstration of building a secure, feature-rich, and extensible VPN client. It combines key principles of software design, cybersecurity, and user interface development into a cohesive product that meets the needs of modern users. By prioritizing security, usability, and scalability, the application sets a strong foundation for future growth and real-world deployment. Whether used as an educational tool, a personal privacy solution, or the basis for a commercial product, KShieldVPN exemplifies the potential of thoughtful and user-centric software development. With continued refinement and expansion, this project has the potential to make a meaningful impact in the realm of online privacy and security.

7.2 Limitations of the system

The current version of the KShieldVPN system, while functional and demonstrative, has several limitations that restrict its effectiveness and scalability in real-world scenarios:

1. Static Server Configuration

   The VPN relies on manually added OpenVPN .ovpn configuration files. Adding new server locations or managing server availability requires manual updates, making scalability difficult.

2. Lack of Advanced User Authentication

   Security measures such as two-factor authentication (2FA), CAPTCHA, and email-based verification are not implemented, which makes the system more vulnerable to unauthorized access.

3. Demo-Based Payment Integration

   Stripe integration functions as a demo without using webhooks to confirm payments. This can allow users to be marked as "premium" without actual payment verification.

4. Limited Session Management

   Session handling is basic and tied to local memory during runtime. Sessions are not encrypted, and there’s no timeout or centralized session tracking for multiple logins.

5. No Kill Switch Functionality

6. The application does not include a kill switch to block internet traffic if the VPN disconnects unexpectedly. This can lead to accidental IP leaks.

7. Simulated Testing Modules

   The DNS leak and speed test modules do not interact with real-time APIs. They are UI-based simulations, which may not provide accurate or actionable feedback to users.

7.3 Future scope of the project

The KShieldVPN project, while successful in its foundational goals, holds vast potential for future development. As digital privacy and secure browsing become increasingly critical, expanding this project into a more advanced, scalable, and user-friendly VPN solution is both timely and feasible. The following points outline the future scope of the system:

1. Cross-Platform Compatibility

   To reach a wider audience, KShieldVPN can be extended to support additional operating systems such as macOS, Linux, Android, and iOS. This will ensure a seamless experience across all devices and make the solution more accessible to mobile and non-Windows users.

2. Intelligent Server Selection

   A smart server selection algorithm based on network latency, load balancing, and geographic proximity can be implemented. This would automatically connect users to the best-performing server, enhancing speed and reliability without requiring manual selection.

3. Real-Time Usage Analytics for Users

   Introducing a dashboard that provides real-time insights on data usage, session duration, IP location, and security stats can increase transparency and help users monitor their activity more effectively.

4. Customizable VPN Protocol Options

   Users can be offered options to switch between different VPN protocols like OpenVPN, WireGuard, and IKEv2, based on their network preferences, performance needs, or device compatibility.

5. Integration with Secure DNS and Ad Blockers
   To enhance privacy and performance, the app can include optional features like DNS over HTTPS (DoH) and ad-blocking at the DNS level, eliminating unwanted traffic and improving browsing speed.

References

1. **How to install Visual Studio 2022 on Windows 11 [https://youtu.be/-B7GObvvems?si=2xZlDsvkdl6sTazX]**
2. **Download Visual Studio Tools - Install Free for Windows, Mac, Linux[https://visualstudio.microsoft.com/downloads/]**
3. **Create Your First C# Windows Forms Application using Visual Studio [https://youtu.be/JSJ1JI2alJg?si=9A0OS7K__A2sALrC]**
4. **How to Download, Install and Activate Guna UI2 in VisualStudio 2022** [https://www.youtube.com/watch?v=0pCoRCS9HVw]
5. **[Hint] How to Create a Complete Forgot Password System | C# [https://youtu.be/4mAS-1-d-cQ?si=qvfM6TvLB5C5pRBJ]**
6. **you need to learn AWS RIGHT NOW!! (Amazon Web Services) [https://youtu.be/bgPuPSPZe2U?si=oczt4rZ37JIWkrsc]**
7. **setup a FREE VPN server in the cloud (AWS) [https://youtu.be/m-i2JBtG4FE?si=8uc_81GCSnxBw0Ly]**
8. **Setup a Free VPN Server using OpenVPN on AWS [https://youtu.be/R7-dj5dvpzA?si=CL8bnUJ4ybL1VM62]**
9. **Git Tutorial For Dummies [https://youtu.be/mJ-qvsxPHpY?si=EUTmO40Eox45UX2A]**
10. **Git and GitHub Tutorial for Beginners [https://youtu.be/tRZGeaHPoaw?si=8_t8C42MPwyFE76X]**
11. **How To Easily Integrate STRIPE Into Your .NET API! [https://youtu.be/2sZXrUFvZ3E?si=xVFoz-9jWDZsWLrK]**
12. **Intermediate ASP.NET Core Tutorial - Online Shop Ep.23 - Stripe Payment Part 1 [https://youtu.be/zFpO0wgrfWY?si=rAGfmNuCcf5gnf32]**
13. **C# Windows Forms: Connect to SQL Server and Perform CRUD Operations | Create Read Update and Delete [https://youtu.be/T8Mq8_Y7tgY?si=4wd2cuTYcjNeTzoM]**